

PAPER • OPEN ACCESS

Performance of parity QAOA for the signed Max-Cut problem

To cite this article: Anita Weidinger *et al* 2026 *New J. Phys.* **28** 034514

View the [article online](#) for updates and enhancements.

You may also like

- [Evaluation of QAOA based on the approximation ratio of individual samples](#)
Jason Larkin, Matías Jonsson, Daniel Justice et al.
- [An expressive ansatz for low-depth quantum approximate optimisation](#)
V Vijendran, Aritra Das, Dax Enshan Koh et al.
- [Modified recursive QAOA for exact MAX-CUT solutions on bipartite graphs: closing the gap beyond QAOA limit](#)
Eunok Bae, Hyukjoon Kwon, V Vijendran et al.



PAPER

Performance of parity QAOA for the signed Max-Cut problem

Anita Weidinger^{1,*} , Glen Bigan Mbeng^{1,2} , Michael Fellner^{1,2} , Davit Khachatryan² 
and Wolfgang Lechner^{1,2,3} 

¹ Institut für Theoretische Physik, Universität Innsbruck, A-6020 Innsbruck, Austria

² Parity Quantum Computing GmbH, A-6020 Innsbruck, Austria

³ Parity Quantum Computing Germany GmbH, 20095 Hamburg, Germany

* Author to whom any correspondence should be addressed.

E-mail: anita.weidinger@uibk.ac.at and wolfgang.lechner@uibk.ac.at

Keywords: quantum optimization, NISQ, quantum computing

RECEIVED

7 October 2025

REVISED

9 February 2026

ACCEPTED FOR PUBLICATION

24 February 2026

PUBLISHED

19 March 2026

Original Content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](https://creativecommons.org/licenses/by/4.0/).

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



Abstract

The practical implementation of quantum optimization algorithms on noisy intermediate-scale quantum devices requires accounting for their limited connectivity. As such, the Parity architecture was introduced to overcome this limitation by encoding binary optimization problems onto planar quantum chips. We investigate the performance of the quantum approximate optimization algorithm on the Parity architecture (Parity QAOA) for solving instances of the signed Max-Cut problem on complete and regular graphs. By comparing the algorithms at fixed circuit depth, we demonstrate that Parity QAOA outperforms conventional QAOA implementations based on SWAP networks. Our analysis utilizes Clifford circuits to estimate lower performance bounds for Parity QAOA for problem sizes that would be otherwise inaccessible on classical computers. For single layer circuits we additionally benchmark the recursive variant of the two algorithms, showing that their performance is equal.

1. Introduction

Given its broad applicability and commercial value, combinatorial optimization is a key area of quantum computation [1, 2]. In this domain, the quantum approximate optimization algorithm (QAOA) [3, 4] recently emerged as a strong candidate for demonstrating quantum advantage on noisy intermediate-scale quantum (NISQ) devices [5–7].

At its core, QAOA is a variational quantum algorithm (VQA) [8] that uses shallow quantum circuits to prepare states, which encode approximate solutions to optimization problems. However, practical implementations of the algorithm on existing NISQ devices face challenges due to limited hardware connectivity and gate error rates [9–11]. To address these issues, various QAOA variants were introduced [4], aiming at reducing the algorithm's execution times and hardware requirements [12–16]. These range from methods to identify hardware-efficient circuits such as QAOA+ [17], ADAPT-QAOA [18] and CD-QAOA to feedback methods that further optimize the quantum states such as recursive QAOA (RQAOA) [19].

Parity QAOA [20–22] is a QAOA variant designed to specifically address qubit connectivity limitations of existing quantum chips. On such chips, conventional QAOA implementations, hereinafter referred to as *vanilla* QAOA, involve SWAP-gates and rerouting strategies which are hard to parallelize and drastically increase the circuit depth [9, 23–32]. In contrast, Parity QAOA distributes the computational load across physical qubits and leverages the chip's planar structure, relying solely on highly parallelizable local gates [21, 33]. In particular, for typical binary optimization problems, parity QAOA requires shallower circuits than vanilla QAOA and, in some instances, also, fewer entangling gates [34]. Moreover, the redundant information distributed across multiple physical qubits is useful for mitigating gate errors on NISQ devices [35].

Despite its advantages, Parity QAOA's planar geometry limits the spread of correlations across the device. This is discussed in [36], where the authors observed that, when solving the Sherrington–Kirkpatrick (SK) spin-glass problem, Parity QAOA's performance deteriorates as the problem size is increased, whereas vanilla QAOA's performance does not. This known phenomenon, which affects QAOA beyond complete graphs, may result in obstacles to preparing high-quality approximate solutions to optimization problems [19].

In this work, we study Parity QAOA to quantify better its advantages and disadvantages over vanilla QAOA. To do so, we compare the algorithm's performances and hardware requirement on the signed Max-Cut problem [37], which includes the SK spin-glass considered in [36] as a special case. We use Clifford pre-optimization to estimate the algorithm's performance and hardware requirement for solving large problem instances of up to 21 nodes, which is beyond the ones considered in previous studies [35, 36]. We find that when a suitable objective function is considered, Parity QAOA consistently outperforms vanilla QAOA at fixed circuit depth. Finally, we address the limited spread of correlation in Parity QAOA's planar geometry by introducing a Parity RQAOA. We show that similarly to its vanilla counterpart, Parity RQAOA overcomes the obstacles to state preparation introduced by the local geometry.

The remainder of this work is organized as follows. First, we introduce the problems of consideration in section 2.1 and review the two different QAOA methods: vanilla QAOA in section 2.2 and Parity QAOA in section 2.3. We then present a procedure to pre-optimize QAOA parameters using Clifford circuits in section 3.1. After that, we present the performance benchmarks of the two QAOA approaches for the signed Max-Cut problem on a complete graph and a 4-regular graph in section 3.2, where we focus on CNOT depth and CNOT count as metrics. Section 3.3 introduces and benchmarks a Parity RQAOA approach. We conclude the manuscript with section 4 by summarizing our findings and presenting an outlook on potential future research directions in section 4.

2. Background

2.1. The Signed Max-Cut problem

This work considers the signed Max-Cut problem, a variation of the classic Max-Cut problem, as introduced in [37], which we define in this section. Possible fields of application for the signed Max-Cut problem are social networks, systems biology, and integrated circuit design [38–41]. This problem is in the strongly NP-hard complexity class [42, 43], therefore, there are no (known) efficient classical optimization algorithms that solve it. In the context of quantum optimization, the signed Max-Cut problem was recently investigated in [44].

Let $G = (V, E)$ be an undirected graph, with vertices V and edges $E \subseteq V^2$. An example of such a graph is depicted in figure 1. Let each edge $(i, j) \in E$ be associated with a weight $J_{ij} \in \{-1, +1\}$, indicating whether the connection between vertices i and j is attractive (+1) or repulsive (−1). The goal in the signed Max-Cut problem is to find a partition of the vertex set V into two disjoint subsets, V_1 and V_2 , such that $V_1 \cup V_2 = V$, with the objective to maximize the sum of the weights of the edges that connect vertices across the two subsets. In other words, we seek a graph bipartition that maximizes the weighted sum of 'cut' edges.

To represent the graph's bipartitions, we associate a bitstring $\mathbf{s} = (s_1, s_2, \dots, s_N) \in \{0, 1\}^N$ with the vertices, where $s_i = 0$ or $s_i = 1$ indicates that vertex i is assigned to subset V_1 or V_2 , respectively, and $N = |V|$. Solving the signed Max-Cut problem equates to minimizing the objective function

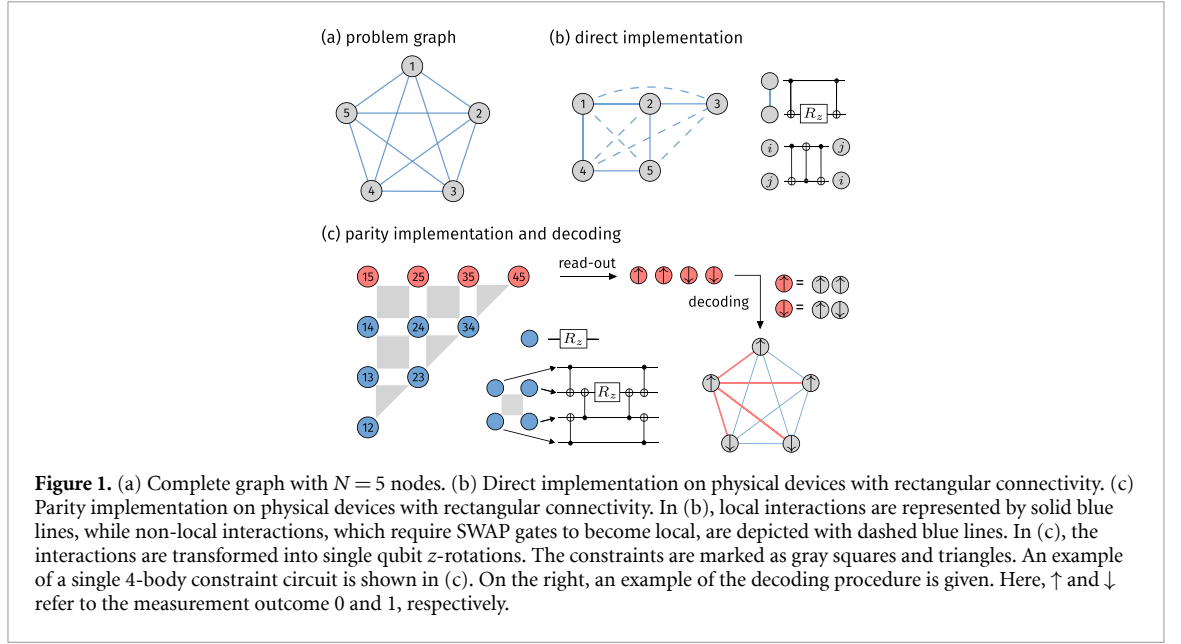
$$C(\mathbf{s}) = - \sum_{(i,j) \in E} J_{ij} (s_i \oplus s_j), \quad (1)$$

which calculates the negative total weight of the edges connecting vertices between the two subsets. Here, $\oplus$ denotes the addition modulo 2. By minimizing this objective function, we effectively maximize the original objective, i.e. the cut of the graph. This type of objective function is not unique to the signed Max-Cut problem. Many combinatorial optimization problems can be cast into this form by mapping them onto a quadratic unconstrained binary optimization (QUBO) problem, often involving arbitrary real weights J_{ij} .

Let $C_{\min}$ and $C_{\max}$ represent the minimum and maximum value of the objective function, respectively, for a given graph. We define the normalized *approximation ratio*

$$\bar{r}(\mathbf{s}) = \frac{C_{\max} - C(\mathbf{s})}{C_{\max} - C_{\min}}, \quad (2)$$

which serves as a metric for evaluating the quality of a proposed solution. This ratio quantifies how close the objective function value $C(\mathbf{s})$ of a particular partition is to the optimal value $C_{\min}$, where



$\bar{r}(s) = 1$ corresponds an optimal solution. For problem instances with non-negative weights $J_{ij} \geq 1$, the maximum value $C_{\max}$ is null and equation (2) corresponds to the familiar expression $\bar{r}(s) = C(s)/C_{\min}$.

Assigning a qubit to each vertex of the graph, the objective function can be encoded in the Hamiltonian

$$H_P = - \sum_{(i,j) \in E} J_{ij} \left(1 - \sigma_i^z \sigma_j^z \right), \quad (3)$$

where σ_i^z represents the Pauli-z operator acting on qubit i . The eigenvalue of σ_i^z corresponds to the assignment of vertex i to one of the two subsets, with eigenvalues $+1$ and -1 indicating membership in V_1 or V_2 , respectively. In the computation basis, H_P coincides with the objective function of equation (1) up to global scaling factor $\frac{1}{2}$. Thus, finding the ground state of H_P is equivalent to finding the optimal partition that maximizes the sum of the signed edge weights between the two subsets, i.e. minimizes equation (1).

We will consider the signed Max-Cut problem on two graphs representing opposite connectivity regimes. In particular, we will present a detailed performance analysis for Parity QAOA's performance on a (fully connected) complete graph in section 3.2.1 and a (sparse) 4-regular graph in section 3.2.2. In addition, we present results for 3-regular graphs in appendix G.1 generalize the problem to hypergraphs in appendix G.3. For all our considerations, we use random weights $J_{ij} \in \{-1, +1\}$.

2.2. QAOA

QAOA [3, 4] is designed to find approximate solutions to combinatorial optimization problems, such as the signed Max-Cut problem.

In vanilla QAOA [3], a phase-separation unitary operator $U_P(\gamma) = e^{-i\gamma H_P}$ is applied alternately with a mixer unitary $U_x(\beta) = \prod_{j=1}^N e^{-i\beta \sigma_j^x}$ p -times to the initial state $|\Psi_0\rangle = |+\rangle^{\otimes N}$ on N qubits. In this context, the integer number p denotes the number of QAOA layers and is chosen before executing the algorithm. The resulting variational state

$$|\Psi(\beta, \gamma)\rangle = U_x(\beta_p) U_P(\gamma_p) \dots U_x(\beta_1) U_P(\gamma_1) |\Psi_0\rangle \quad (4)$$

after p layers depends on the $2p$ parameters $\gamma = (\gamma_1, \dots, \gamma_p)$ and $\beta = (\beta_1, \dots, \beta_p)$. A classical optimization routine is used within a quantum-classical feedback loop to find the parameters (β^*, γ^*) that minimize the objective function

$$\mathcal{C}(\beta, \gamma) = \langle \Psi(\beta, \gamma) | H_P | \Psi(\beta, \gamma) \rangle. \quad (5)$$

The objective function is estimated by repeatedly preparing and measuring the variational state $|\Psi(\beta, \gamma)\rangle$. For $p \rightarrow \infty$, the optimal value $\mathcal{C}(\beta^*, \gamma^*)$ converges monotonically to the ground state energy of H_P and $|\Psi(\beta^*, \gamma^*)\rangle$ converges to the optimization problem's solution [3, 45]. For finite p , measuring $|\Psi(\beta^*, \gamma^*)\rangle$ will provide approximate solutions. This implies that it is theoretically possible to

find the exact solution for a problem for sufficiently large p . However, in practice, one often seeks an approximate solution and therefore limits p to maintain a feasible optimization space. Then, the solutions obtained via vanilla QAOA, have the approximation ratio

$$r = \frac{C_{\max} - \mathcal{C}(\beta^*, \gamma^*)}{C_{\max} - C_{\min}}. \quad (6)$$

Due to the limited hardware connectivity of many current quantum devices, a direct implementation of $U_P(\gamma)$ can be challenging if the interaction graph E is not well matched to the device topology, as it may involve non-local entangling gates between distant qubits. While local interactions can be efficiently implemented using either native two-qubit gates or two CNOT gates and a single-qubit z -rotation, non-local interactions on connectivity-limited platforms typically necessitate a rerouting strategy [23, 31] involving additional SWAP gates (each comprising three CNOT gates) to establish the required interactions (see figure 1(b)). Such routing overhead and the required SWAP networks are highly problem- and hardware-dependent and may, for unfavorable graph-hardware mappings, result in undesired circuit depth overhead for vanilla QAOA, which scales linearly with the number of qubits N [30–32]. In addition, if SWAP gates are not efficiently arranged, their excessive use can lead to unnecessary noise and degrade the accuracy of the computation.

We note that some quantum processors include two-qubit R_{zz} interactions in their native gate set. Nevertheless, non-local interaction can still introduce substantial overhead on devices with restricted connectivity, motivating approaches—such as Parity QAOA discussed in section B—that reduce (or even avoid) long-range couplings.

Several compilers [27–29, 46] provide algorithms to optimize the circuit depth and gate count of vanilla QAOA on quantum hardware with linear [9, 23, 30] or square-grid connectivity [31]. For instance, when compiling complete graphs on a square-grid device, using linear SWAP networks is the optimal strategy, resulting in a CNOT depth⁴ of $3N + 4$ and gate count of $\frac{3}{2}N(N - 1)$ per vanilla QAOA layer [30, 31].

2.3. Parity quantum approximate optimization

The parity transformation [20, 22] is an alternative solution for the limited connectivity of the quantum hardware by mapping multi-qubit entangling terms to single-qubit terms. The transformation replaces the original N logical qubits σ_j^z with K physical qubits $\tilde{\sigma}_\nu^z$, such that each physical qubit represents the parity of a pair of logical qubits $\sigma_{i_\nu}^z \sigma_{j_\nu}^z \rightarrow \tilde{\sigma}_\nu^z$. Hereafter, we will mark physical states and operators with a tilde to distinguish them from logical ones.

Starting from equation (3), and applying the parity transformation results in the local Hamiltonian

$$\tilde{H}_P = \sum_{\nu=1}^K \tilde{J}_\nu \tilde{\sigma}_\nu^z \quad (7)$$

for the physical qubits, where $\tilde{J}_\nu = J_{i_\nu j_\nu}$ with $\nu = 1, 2, \dots, K$ are local fields for the physical qubits. In particular, the problem of finding the ground state of the original Hamiltonian H_P is equivalent to the problem of finding the minimum energy state of the physical Hamiltonian $\tilde{H}_P$ with $L = K - N + D$ independent constraints, where D denotes the number of logical degeneracies of H_P (see [22] for details). The role of these constraints is to enforce that all considered configurations of physical qubits correspond to a unique configuration of logical qubits. Lechner *et al* [20] showed that this condition can be attained by arranging the qubits on a square-grid planar geometry and choosing the set of L local constraints to be

$$\tilde{H}_{\blacksquare, l} |\tilde{\psi}\rangle = |\tilde{\psi}\rangle \quad \text{for } l = 1, 2, \dots, L, \quad (8)$$

such that $\tilde{H}_{\blacksquare, l} = \tilde{\sigma}_{(l,1)}^z \tilde{\sigma}_{(l,2)}^z \tilde{\sigma}_{(l,3)}^z [\tilde{\sigma}_{(l,4)}^z]$ are local four- and three-qubit plaquette terms on the grid. The correct choice for these constraint terms depends on the original optimization problem. In particular, complete problem graphs are known to require $K = \frac{N(N-1)}{2}$ physical qubits and $L = \frac{(N-1)(N-2)}{2}$ constraints arranged in a Lechner–Hauke–Zoller geometry (see figure 1(b)) [20]. Recently, Ender *et al* [22] extended the parity transformation to Polynomial Unconstrained Boolean Optimization (PUBO) problems by encoding the parity of multiple logical qubits in a single physical qubit and designing efficient algorithms to identify the correct geometry for the physical qubits and the plaquette terms [47].

⁴ We define the CNOT depth as the number of circuit layers comprising at least one CNOT gate.

In Parity QAOA [21, 35], two phase-separation unitary operators $\tilde{U}_z(\gamma) = e^{-i\gamma\tilde{H}_p}$ and $\tilde{U}_{\square}(\Omega) = \prod_{l=1}^L e^{-i\Omega\tilde{H}_{\square,l}}$ are applied alternately with a mixer unitary $\tilde{U}_x(\beta) = \prod_{\nu=1}^K e^{-i\beta\tilde{\sigma}_j^x}$ p -times to the initial state $|\tilde{\Psi}_0\rangle = |+\rangle^{\otimes K}$ on K qubits. Adding another set of variational parameters $\Omega = (\Omega_1, \dots, \Omega_p)$ for the constraint unitaries $\tilde{U}_{\square}$ results in better performance than merging it with the main cost Hamiltonian into a single phase operator, as demonstrated in [21]. The resulting variational state for Parity QAOA is

$$|\tilde{\Psi}(\beta, \Omega, \gamma)\rangle = \tilde{U}_x(\beta_p) \tilde{U}_{\square}(\Omega_p) \tilde{U}_z(\gamma_p) \dots \dots \tilde{U}_x(\beta_1) \tilde{U}_{\square}(\Omega_1) \tilde{U}_z(\gamma_1) |+\rangle^{\otimes K}, \quad (9)$$

Lechner [21] showed that, for $p \rightarrow \infty$, this state can represent the minimum energy state of $\tilde{H}_p$ with the constraints in equation (8), which corresponds to the original optimization problem's solution.

Parity QAOA is particularly suited for quantum hardware with planar square-grid connectivity. Therefore, we consider square-lattice hardware platforms for this study. Indeed, implementing $\tilde{U}_z(\gamma)$ and $\tilde{U}_x(\beta)$ only requires composing single-qubit gates, with no hardware connectivity requirements. On the other hand, the four-qubit (three-qubit) plaquette gates $e^{-i\Omega\tilde{H}_{\square,l}}$ can be implemented with a chain of six (four) CNOT gates and a single-qubit z -rotation (see figure 1(c)). All constraint terms can be implemented in parallel, resulting in a constant circuit depth per QAOA layer [21, 33]. Specifically, for a QUBO problem on a complete graph, the CNOT depth is 10 (independent of N), and the gate count per QAOA layer is $2(N-2)(N-3)$ on a square grid [33]. The CNOT depth and gate count per layer for other graphs can be obtained by following the procedure in [33].

To run Parity QAOA, we also need to specify a measurable objective function that is minimized when the optimization problem is solved. The original Parity QAOA objective function, introduced in [21], uses penalty terms to enforce the plaquette constraints of equation (8). However, as pointed out in [35], the algorithm performs better when the projective measurements of the physical K -qubit variational state are explicitly decoded into classical configurations of the N logical variables.

Following [35], we thus employ a decoding strategy that assigns values $\mathbf{q} = (q_1, \dots, q_N) \in \{0, 1\}^N$ to N logical qubits based on a given readout measurement of a subset of the K physical qubit measurement outcome $\tilde{\mathbf{q}} = (\tilde{q}_1, \dots, \tilde{q}_K)$. We use the term ‘readout basis’ to refer to these subsets of physical qubits. On a graph with a single logical degeneracy ($D=1$), the readout basis of the physical qubits represents the edges of a spanning tree t in the logical graph; that is, the corresponding edges form a tree that includes all nodes of the problem graph. An example of a single tree is given in figure 1(c) on the right. A spanning tree contains $N-1$ edges, hence, $N-1$ parity qubits must be in the readout basis. The measurement outcome of a parity qubit describes the parity between two logical values. A readout of 0 means that the corresponding logical qubit values are identical (e.g. 0 and 0) and, conversely, if the readout is 1 the logical qubits values are different (e.g. 0 and 1). For each spanning tree t , applying these rules to all $N-1$ readout variables, we can determine a decoded logical state $\mathbf{q}^{(t)}$ (up to a global spin flip). Different spanning trees can lead to different logical values. For general graphs and hypergraphs, readout bases have $N-D$ physical qubits and are not necessarily spanning trees [22].

We define the projector

$$P_{\mathbf{q}^{(t)}} = |\mathbf{q}^{(t)}\rangle\langle\mathbf{q}^{(t)}| \quad (10)$$

onto the decoded logical state $\mathbf{q}^{(t)}$. A decoding map D_t can then be defined as

$$D_t \left[\tilde{\Psi}(\beta, \Omega, \gamma) \right] = \sum_{\tilde{\mathbf{q}} \in \{0,1\}^K} |\tilde{\mathbf{q}}| \tilde{\Psi}(\beta, \Omega, \gamma) \rangle \langle \tilde{\mathbf{q}}| P_{\mathbf{q}^{(t)}}. \quad (11)$$

In this description, the measurement outcome $\tilde{\mathbf{q}}$, which appears with probability $|\langle\tilde{\mathbf{q}}|\tilde{\Psi}(\beta, \Omega, \gamma)\rangle|^2$, is translated to a logical configuration $\mathbf{q}^{(t)}$.

Considering a set $T = \{t_1, t_2, \dots, t_M\}$ of M spanning trees used for decoding, we can define the objective function as

$$\tilde{C}^{\text{mean}}(\beta, \Omega, \gamma) = \frac{1}{M} \sum_{m=1}^M \text{Tr} \left(H_P D_{t_m} \left[\tilde{\Psi}(\beta, \Omega, \gamma) \right] \right), \quad (12)$$

which corresponds to the mean energy over the different spanning tree readouts $\mathbf{q}^{(t_i)}$ with $i = 1, \dots, M$. An alternative way to define the objective function is to consider the best (lowest) energy of the M spanning tree readouts

$$\tilde{C}^{\text{best}}(\beta, \Omega, \gamma) = \min_{t \in T} \text{Tr} \left(H_P D_t \left[\tilde{\Psi}(\beta, \Omega, \gamma) \right] \right). \quad (13)$$

Then, the approximation ratio's expected value for the solutions obtained via Parity QAOA is

$$r = \frac{C_{\max} - \tilde{C}(\beta^*, \Omega^*, \gamma^*)}{C_{\max} - C_{\min}}, \quad (14)$$

where $\tilde{C}(\beta^*, \Omega^*, \gamma^*)$ can either be the mean $\tilde{C}^{\text{mean}}$ or the best spanning tree energy $\tilde{C}^{\text{best}}$, as defined in equations (12) and (13).

For a complete graph, N^{N-2} spanning trees exist. Weidinger *et al* [35] showed that the performance of Parity QAOA increases the more spanning trees are used but does not change significantly for a particular choice of trees. Following [35], we limit the classical decoding overhead by considering a specific set of $M = N$ spanning trees, namely the *logical lines* in the parity encoding, denoted by $T^{(l)} = \{t_1^{(l)}, t_2^{(l)}, \dots, t_N^{(l)}\}$. The logical line i , corresponding to spanning tree $t_i^{(l)}$, includes all parity qubits that contain information about the logical qubit i . (i.e. all qubits $\tilde{\sigma}_\nu$ that encode the parity of the logical qubits σ_i and σ_j where $j = 1, \dots, N$, excluding $j = i$). This means that for a complete graph with N nodes there are exactly N logical lines. Figure 1(c) shows a spanning tree readout of $t_5^{(l)}$.

2.4. Clifford pre-optimization

The term Clifford circuit pre-optimization describes a classical strategy to find high-quality (initial) parameters before running a VQA (e.g. QAOA) on a quantum device [48, 49]. This section introduces Clifford gates and their use in QAOA pre-optimization. More information on the topic can be found in [48–50].

The defining property of Clifford gates is mapping Pauli matrices and their tensor products into themselves by conjugation. More precisely, let $\mathcal{P}_N$ be the N -qubit Pauli unitary group generated by the single qubit operations $\{\sigma_j^x, \sigma_j^y, \sigma_j^z\}_{j=1, \dots, N}$. Then, N -qubit Clifford gates are unitary transformations G such that $G^\dagger X G \in \mathcal{P}_N$ for all $X \in \mathcal{P}_N$. Their relevance in QAOA follows from the Gottesman–Knill theorem [50–52], which ensures that Clifford gates can be simulated efficiently (in polynomial time) on classical computers.

For the signed Max-Cut problem class, the vanilla QAOA unitaries $U_\kappa(\theta)$ with $\kappa = x, P$ and the Parity QAOA unitaries $\tilde{U}_\kappa(\theta)$ with $\kappa = x, \square, P$ are Clifford gates if $\theta \in A = \{-\frac{\pi}{4}, 0, \frac{\pi}{4}, \frac{\pi}{2}\}$ ⁵. Therefore, when $\beta, \Omega, \gamma \in A^p$, we can use the Gottesman–Knill theorem to efficiently sample the vanilla and Parity QAOA objective functions on a classical computer. The Clifford circuit pre-optimization is a search aimed at (locally or globally) minimizing the QAOA objective function, with the constraint $\beta, \Omega, \gamma \in A^p$. This approach's main advantage is that the search algorithm can run efficiently on a classical computer with a query cost polynomial in the problem size N .

Let $\beta^{(0)}, \Omega^{(0)}, \gamma^{(0)} \in A^p$ be the optimal parameters found by the Clifford circuit pre-optimization. These parameters can be used as initial values for QAOA. In this case, QAOA's approximation ratio is bounded by $r > r^{(0)}$, where $r^{(0)}$ is the approximation ratio associated with the found optimal Clifford circuit:

$$r^{(0)} = \frac{C_{\max} - \tilde{C}(\beta^{(0)}, \Omega^{(0)}, \gamma^{(0)})}{C_{\max} - C_{\min}}. \quad (15)$$

The value of $r^{(0)}$ thus provides a classical lower bound (LB) to the accuracy of the QAOA algorithm. This LB is particularly useful for predicting QAOA's performance on large problem sizes, which cannot be simulated on classical computers.

⁵ This statement follows from the observation that for any non-trivial Hermitian Pauli string $X \in \mathcal{P}_N$ the exponential $e^{i\theta X}$ is a Clifford gate if and only if θ is an integer multiple of $\pi/4$. Moreover, since $e^{-i\theta X} Y e^{i\theta X}$ has a period of 2π , we only need to consider the values $\theta \in \{-\frac{\pi}{4}, 0, \frac{\pi}{4}, \frac{\pi}{2}\}$.

Table 1. Classical parameter vectors for even QAOA layers $p > 2$. Values for arbitrary p can be found in appendix A. The table shows the index values $l = 1, \dots, p$, for the parameters $(\gamma_l, \Omega_l, \beta_l)$ (which are given in units of π).

l	γ_l	Ω_l	β_l
$1, \dots, \frac{p-2}{2}$	0	$\frac{1}{4}$	$\frac{1}{4}$
$\frac{p}{2}$	0	$\frac{1}{4}$	$\frac{1}{2}$
$\frac{p+2}{2}, \dots, (p-1)$	$0/\frac{1}{2}$	$\pm \frac{1}{4}$	$\frac{1}{4}$
p	$\pm \frac{1}{4}$	$\pm \frac{1}{4}$	$\frac{1}{4}$

3. Results

We present data from numerical simulations that aim to compare vanilla and Parity QAOA's performances on the signed Max-Cut problem. We focus on complete and 4-regular graphs, using the circuits' CNOT depth and CNOT counts to quantify the hardware resources required to run the two algorithms on a square-grid architecture. We conclude the section by introducing a recursive variant Parity QAOA variant and comparing it with the conventional RQAOA approach. In appendix H we present additional results for linear-ramp QAOA [53], another QAOA variant, in which the classical optimization loop is omitted and the QAOA variational parameters are instead chosen according to a linear schedule.

3.1. Performance lower bounds for parity QAOA

The quadratic qubit overhead introduced by parity QAOA makes simulating the algorithm on large graphs ($N > 20$) a challenging task. Therefore, we use Clifford pre-optimization to establish lower bounds on Parity QAOA's performance, which we will later compare with vanilla QAOA's results (see section 3.2).

We can restrict the optimal parameter search to the subset $A_{\text{class}}^{(p)} \subset A^p \times A^p \times A^p$ to further reduce the Clifford pre-optimization's running time (for a detailed discussion see appendix A). Here, we chose $A_{\text{class}}^{(p)}$ such that for $(\gamma, \Omega, \beta) \in A_{\text{class}}^{(p)}$ the prepared parity QAOA state $|\tilde{\Psi}(\beta, \Omega, \gamma)\rangle$ is an element of the computational basis $\{|\tilde{q}_1, \dots, \tilde{q}_K\rangle \mid \tilde{q}_i \in \{0, 1\}\}$. The elements of $A_{\text{class}}^{(p)}$ for the signed Max-Cut problem, are explicitly given in table 1. We refer to them as *classical parameter vectors* and to the corresponding states as *classical states* in the following. The number of classical parameter vectors increases exponentially with the number of layers p , following the relation $|A_{\text{class}}^{(p)}| = 2^{p+1}$. We refer to appendix A for more details about these classical parameter vectors.

We now briefly describe the procedure we used to compute the lower bounds on the approximation ratio for Parity QAOA applied on the signed Max-Cut problem on a complete graph. For each classical parameter vector, we use the python library *stim* [54] to efficiently simulate the Parity QAOA circuit and compute the output state. As each circuit returns a classical state, which is an element of the computational basis, no sampling is required. We, then, decode the hereby obtained classical states using the spanning trees $T^{(l)}$ as described in section 2.3. After evaluating the objective function for the resulting state according to equation (12) or equation (13), we use the lowest measured energy of the 2^{p+1} states to compute the LB on the approximation ratio. The complexity of the whole procedure scales exponentially with the QAOA layers p .

Figure 2 shows the results for the approximation ratio, as defined in equation (14), as a function of the system size N , averaged over randomly generated instances (see figure caption). An approximation ratio of $r = 1$, marked with a black dashed line, signifies that the exact solution (configuration with the lowest objective function) is found.

The results indicate that choosing the best spanning tree energy $\tilde{C}^{\text{best}}$ (best, $p = 1$) yields a significantly higher approximation ratio than taking the mean $\tilde{C}^{\text{mean}}$ (mean, $p = 1$). In fact, using the objective function given by equation (13) and $p = 1$ QAOA layer yields an approximation ratio of $r = 1.0$ for $N = 4$, indicating that all problem instances were solved. For $p = 10$ and $\tilde{C}^{\text{best}}$, the classical parameter vectors generate the solution for every instance up to $N = 12$. In contrast to that, using the mean energy $\tilde{C}^{\text{mean}}$ does not show significant improvement when increasing the number of layers from $p = 1$ to $p = 10$.

In appendix C we elaborate more on the success rate of decoding classical states. Additionally, appendix F.1 presents results for QAOA simulations for both the best $\tilde{C}^{\text{best}}$ and mean $\tilde{C}^{\text{mean}}$ objective function. In the following sections we choose the best spanning tree energy $\tilde{C}^{\text{best}}$ as the objective function.

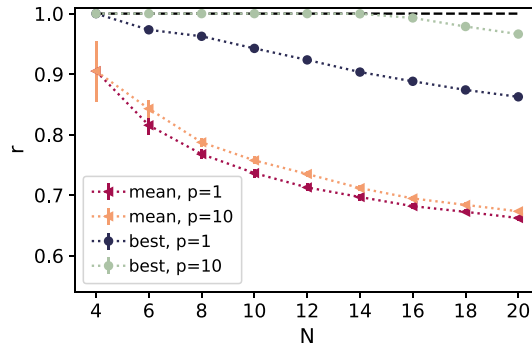


Figure 2. The approximation ratio r as a function of the logical qubit number N for the objective functions $\tilde{C}^{\text{mean}}$ (red and orange line, equation (12)), and $\tilde{C}^{\text{best}}$ (blue and green line, equation (13)), for the decoded classical states with $M = N$ spanning trees. Data points represent averages over 50, 400, and 800 instances for $N = 4, 6$ and $N > 6$, respectively, with error bars showing twice the standard error of the mean.

We note that we did not attempt a similar analysis for vanilla QAOA, as [49] observed that Clifford pre-optimization for Max-Cut problems does not yield meaningful lower bounds for vanilla QAOA. Nevertheless, there are cases where Clifford gates play a significant role in vanilla QAOA [48, 55, 56].

3.2. QAOA simulations

Next, we investigate the performance of vanilla and Parity QAOA with respect to CNOT gate count and circuit depth for the signed Max-Cut problem on a complete and a 4-regular graph. Vanilla QAOA simulations are done with K/N copies to equalize the number of qubits required in both embeddings. QAOA is executed on all copies in parallel, where, after measurement of all qubits, the lowest energy of all copies is considered to evaluate the objective value. However, we do not multiply the CNOT depth or gate count by the K/N factor; instead, we compare the hardware requirements to run the individual QAOA circuits.

Different QAOA variants can require significantly different hardware costs per layer. Therefore, we present our comparisons both as a function of the hardware resources (circuit depth, gate count), while the number p of QAOA layers can be inferred in that the p -th point in a data set corresponds to a QAOA implementation with p layers.

The initial parameters for the classical optimization for both QAOA methods are chosen randomly from the interval $[-\frac{\pi}{2}, \frac{\pi}{2})$. In particular, our QAOA simulations do not use the Clifford pre-optimization described in section 2.4 to enhance the algorithms' performance. More details are provided in appendix E.

The QAOA circuit is simulated with the Python library *qiskit* [46]. Furthermore, we compare the QAOA simulation results with the lower bounds, which were computed in the last section.

3.2.1. Complete graph

We start by analyzing the performance of Parity QAOA for signed Max-Cut on a complete graph, i.e. the SK spin glass. Figure 3(a) shows the average approximation ratio for 100 random instances with $N = 7$ nodes, for both vanilla and Parity QAOA. The approximation ratio is evaluated with respect to the CNOT depth on the left and the CNOT gate count on the right. The circuit depth and the gate count for p QAOA layers in both approaches were obtained from the numbers for a single layer discussed in sections 2.2 and 2.3.

Each data point corresponds to a specific QAOA depth p : the left-most point represents one layer ($p = 1$), the second left-most point represents two layers ($p = 2$), and so on. The blue dotted line indicates the LB presented in section 3.1. When QAOA is applied, the approximation ratio exceeds the LB.

The data points for the LB and the actual optimization results lie close to each other because decoding the Parity QAOA output state yields the ground state for most instances, e.g. for 96 out of 100 for $p = 3$. We refer to these instances as *Clifford-solvable* instances. In contrast, instances not exactly solvable by decoding the classical states alone are termed *Clifford-non-solvable* instances. In figure 4, we present results obtained by simulating vanilla and Parity QAOA and averaging the approximation ratio over a set of 30 postselected Clifford-non-solvable instances of $N = 7$.

The figure illustrates that, for Clifford-non-solvable instances, the application of Parity QAOA improves the approximation ratio beyond the LB obtained by Clifford pre-optimization. A more detailed

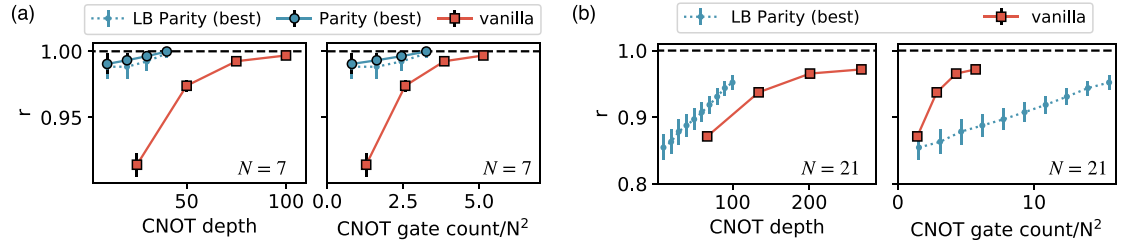


Figure 3. Approximation ratio r for vanilla and Parity QAOA as a function of CNOT depth and CNOT gate count. In each line, the left-most data point represents $p = 1$, the second left-most data point represents $p = 2$, and so forth. (a) Results for $N = 7$ logical qubits ($K = 21$ physical qubits). Dotted lines indicate the lower bound (LB) results obtained by decoding classical states. Classical optimization parameters are: $n_{\text{init}} = 80$, $n_{\text{mc}} = 400/1000/1500/2000$ for $p = 1/2/3/4$. Results represent the average over 100 simulated instances. (b) Results for $N = 21$ ($K = 210$ physical qubits). Due to the system size, simulation of Parity QAOA is not feasible; therefore, only the LB is shown. Classical optimization parameters for vanilla QAOA are $n_{\text{init}} = 50$ and $n_{\text{mc}} = 200/500/800/1000$ for $p = 1/2/3/4$. For the LB of Parity QAOA, data is shown for higher p than for vanilla QAOA due to the feasibility of the simulations. Results represent the average over 40 simulated instances. In both plots, the error bars represent twice the standard error of the mean.

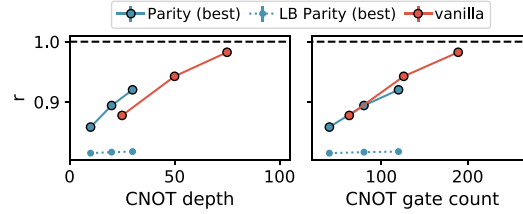


Figure 4. Approximation ratio r of Parity and vanilla QAOA for the complete graph of Clifford-non-solvable instances for $p = 3$ and $N = 7$ in dependence of CNOT depth and CNOT gate count. In each line, the left-most data point represents $p = 1$, the second left-most data point represents $p = 2$, and so forth. The data points show the average of 30 Clifford-non-solvable instances. The error bars indicate two times the standard error of the mean and are not visible due to the size of the markers. Classical optimization parameters are: $n_{\text{init}} = 80$, $n_{\text{mc}} = 400/1000/1500/2000$ for $p = 1/2/3/4$.

Table 2. Resources needed for Vanilla (V) and Parity QAOA (P) to reach an approximation ratio of $r > 0.95$ for the three different problem cases shown. For Parity QAOA the numbers for the complete graph (SK), $N = 21$ are provided by the Lower Bound (LB).

	SK, $N = 7$		SK, $N = 21$		4-reg, $N = 8$	
	P	V	P (LB)	V	P	V
p	1	2	10	3	1	3
depth	10	50	100	201	12	60
gate count N^2	0.82	2.57	15.51	4.29	0.69	1.78
no. of qubits	21	21	210	210	17	16
no. of read-outs	7	3	21	10	5	2

discussion of Clifford-non-solvable instances, together with an in-depth analysis of the postselected instances, is provided in appendix F.

In terms of CNOT depth, Parity QAOA consistently achieves a higher approximation ratio for $N = 7$. This behavior persists when N increases, as shown in figure 3(b), which presents results for $N = 21$. Simulations for this scenario, involving $K = 210$ physical qubits, can not be simulated on a classical computer. Thus, only the LB of the approximation ratio is reported. While the Parity QAOA LB shows an advantage in terms of CNOT depth (left), vanilla QAOA is superior in terms of CNOT gate count (right). This is expected, as the depth for one layer of Parity QAOA is independent of the system size, while it scales linearly with N for vanilla QAOA. In leading order, the CNOT gate count scales with $2N^2$ for Parity QAOA and $\frac{3}{2}N^2$ for vanilla QAOA.

Appendix D provides a more detailed discussion on the differences in implementation, qubit overhead and required post-processing resources of the two QAOA approaches. A comparison of explicit numbers for different metrics to reach $r > 0.95$ is given in table 2. Note that for $N = 21$, we can only provide the LB for parity QAOA. Number of readouts refer to the number of copies (vanilla) and the number of spanning trees M (parity) used.

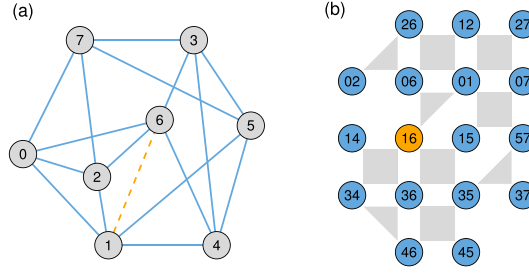


Figure 5. (a) 4-regular graph in the logical representation. The interaction corresponding to the physical ancilla qubit (16) is shown as a yellow dashed line. (b) Compiled parity graph obtained from the 4-regular graph shown in (a), with the ancilla qubit (16) colored yellow.

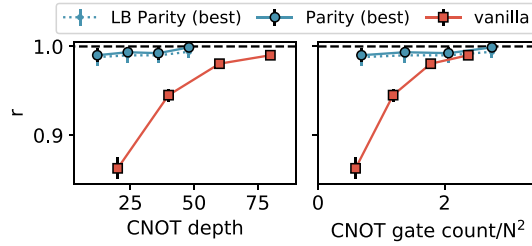


Figure 6. Approximation ratio r for vanilla and Parity QAOA as a function of CNOT depth and CNOT gate count for a 4-regular graph with $N = 8$ nodes and $M = 5$. In each line, the left-most data point represents $p = 1$, the second left-most data point represents $p = 2$, and so forth. The dotted lines show the LB of the approximation ratio. Classical optimization parameters are: $n_{\text{init}} = 80$, $n_{\text{mc}} = 400/1000/1500/2000$ for $p = 1/2/3/4$. Results show the average over 100 simulated instances. The error bars represent twice the standard error of the mean.

3.2.2. 4-regular graph

The parity transformation can also be done for non-complete graphs [22]. An arbitrary example, with lower connectivity and small enough to classically simulate, is the 4-regular graph with $N = 8$ nodes shown in figure 5. The parity graph was obtained by the ParityOS compiler [22, 47], which introduced the ancilla qubit (16) to complete the mapping. To compare the performance of Parity QAOA on such *compiled graphs*, we again consider 100 random choices of the weights $J_{ij} \in \{-1, 1\}$.

For compiled graphs not all edges are present and, therefore, the $M = N$ logical lines as described before do not exist. However, like in [35], we choose a random set of spanning trees. Regarding the number of trees, we could, in principle, consider a set $T = \{t_1, t_2, \dots, t_8\}$ of $M = N = 8$ spanning trees, that uses the same number of spanning trees as the complete graph. Considering the fact that the compiled graph has less physical qubit overhead than the complete graph one can also choose a set with fewer spanning trees, such that the average number of spanning trees per physical qubit D_M equals the number of logical lines for the complete graph. One can calculate D_M as follows. A spanning tree covers $N - 1$ parity qubits, and with M spanning trees over K parity qubits we obtain the relation $D_M = M(N - 1)/K$. In the complete graph, we have $D_M = 2$ for $M = N$, which means that each parity qubit is contained in 2 spanning trees. To obtain a similar value in the 4-regular graph, whose parity mapping is depicted in figure 5(b), we require $M = 5$ spanning trees, $T = \{t_1, \dots, t_5\}$, yielding $D_M = 2.06$. In appendix F.2, we show a comparison of the two choices of spanning trees (with $M = 8$ and $M = 5$, see figure 12(b)) and give the explicit spanning trees we used for decoding. In the main text, we will only consider the set with $M = 5$ spanning trees.

Following the constraint implementation procedure in [33], the Parity QAOA circuit has a CNOT depth of 12 and a CNOT-gate count of 44. The circuit for the vanilla QAOA was transpiled by t|ket> [27] and then further optimized. The resulting circuit has a CNOT depth of 20 and a CNOT gate count of 38. Results for QAOA simulations for 100 problem instances are shown in figure 6. Again, due to the high LB for the success rate of Parity QAOA, the approximation ratio reaches $r > 0.97$. There are 6/5/3/3 Clifford-non-solvable instances for $p = 1/2/3/4$ for 100 simulated instances. These results are in accordance with the data obtained for the complete graph. This also holds for larger 4-regular graphs ($N = 14$), shown in appendix G.2. Simulations for exclusively Clifford-non-solvable instances are discussed in appendix F.2.

3.3. RQAOA simulations

QAOA's performance and limitations on regular graphs were studied in [19, 36, 45, 57]. The authors showed that QAOA needs to 'see the whole graphs' to avoid the obstacles arising from the algorithm's locality. In particular, Farhi *et al* [57] proved that approximating the solutions of typical random regular graphs requires scaling the number of QAOA layers with the graph size at least logarithmically. Wybo and Leib [36] extended these results to the Parity architecture, showing that approximating the solutions of complete graphs with Parity QAOA requires a number of layers that scales at least as $\mathcal{O}(\sqrt{N})$.

Bravyi *et al* [19] proposed a RQAOA to solve Max-Cut problems to overcome the performance limitations induced by locality. Starting from the original N -qubit problem Hamiltonian $H_p^{(N)} = H_p$, the RQAOA iteratively reduces the problem's size by a procedure called *correlation rounding* [19, 58]. Given an N' -qubit Hamiltonian $H_p^{(N')}$, a single variable elimination via correlation rounding works as follows: First we use the QAOA subroutine to compute the state $|\Psi^{(N')}(\beta^*, \gamma^*)\rangle$ that best approximates the ground state of $H_p^{(N')}$. Then we use repeated measurements to estimate the correlation matrix

$$W_{jk}^{(N')} = \langle \Psi^{(N')}(\beta^*, \gamma^*) | \sigma_j^z \sigma_k^z | \Psi^{(N')}(\beta^*, \gamma^*) \rangle. \quad (16)$$

Next, we compute the edge with the largest absolute correlations

$$(\bar{j}, \bar{k}) = \underset{(j,k) \in E}{\operatorname{argmax}} \left| W_{jk}^{(N')} \right|. \quad (17)$$

Finally, we reduce the problem size by the fixing rule

$$\sigma_j^z = \operatorname{sign} \left(W_{\bar{j}, \bar{k}}^{(N')} \right) \sigma_{\bar{k}}^z. \quad (18)$$

substituting into $H^{(N')}$. The substitution results in a new $(N' - 1)$ -qubit problem Hamiltonian $H^{(N'-1)}$. The variable-elimination process is repeated on the increasingly smaller graph until the problem becomes trivial to solve. Then, the sequence of fixing rules is reversed to obtain an approximate solution for the original problem.

The non-local operations involved in RQAOA's correlation rounding can overcome QAOA's performance limitations [58]. In particular, several numerical and theoretical studies [58–60] suggest that using RQAOA leads to finding better approximate solutions with performances that are comparable to state-of-the-art general-purpose classical solvers. In the following, we introduce a parity-based implementation of RQAOA by making suitable changes to the original RQAOA algorithm. We use the terms vanilla RQAOA and Parity RQAOA to distinguish between the original and the new parity-based RQAOA versions.

To implement Parity RQAOA, we replace the correlation matrix of equation (16) with

$$W_{jk}^{(N')} = \frac{1}{M} \sum_{m=1}^M \operatorname{Tr} \left(\sigma_j^z \sigma_k^z D_{t_m} \left[\tilde{\rho}^{(N')}(\beta^*, \Omega^*, \gamma^*) \right] \right). \quad (19)$$

Then, we use correlation rounding to iteratively eliminate logical variables from the original problem equation (18).

Although analytical results on Parity RQAOA's performance are not available, we can use numerical simulations to assess the algorithm's performance. We study Parity RQAOA's performance numerically for $p = 1$ layer, where analytical formulas for the objective function are available. Following [36], we consider different instances of the signed Max-Cut problem on a complete graph with random weights $J_{jk} \in \{-1, 1\}$ (see section 3.2.1). In figure 7, we compare single-layer QAOA and RQAOA approximation ratios resulting from a Parity implementation with those resulting from a vanilla implementation, where we use $\tilde{C}^{\text{mean}}$ as an objective function for Parity QAOA. As predicted in [36], Parity QAOA produces approximation ratios below vanilla QAOA. However, using RQAOA results in higher approximation ratios for both implementations. In particular, Parity RQAOA performs as well as vanilla RQAOA, despite the latter requiring deeper circuits.

4. Conclusion and outlook

In this work, we studied the performance and characteristics of Parity QAOA and compared them with those of vanilla QAOA. The analysis focused on the signed Max-Cut problem for the SK spin glass and

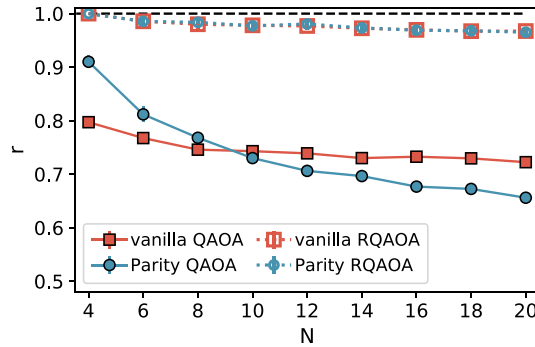


Figure 7. Approximation ratio r for single layer RQAOA ($p = 1$) on complete graphs. Vanilla RQAOA (red empty squares) and parity RQAOA (blue empty circles) approximation ratios are shown as a function of the graph size N . The filled symbols correspond to the single-layer QAOA simulations on the same problem graph. The objective function $\tilde{C}^{\text{mean}}$ (equation (12)) was used for the Parity architecture. Data points represent averages over 100 simulated instances. The error bars are twice the standard error of the mean.

random regular graph instances, leveraging Clifford circuits to estimate Parity QAOA's performance on large systems.

Our QAOA numerical simulations highlight how the choice of the decoding strategy affects Parity QAOA's performance. In particular, we found that selecting an optimal spanning tree during each measurement decoding significantly enhances the algorithm's performance. Taking this into account, we verified that Parity QAOA outperforms vanilla QAOA by achieving higher approximation ratios with a smaller circuit depth on problems with up to twenty-one variables.

For larger systems, the local nature of interactions in the Parity architecture is expected to hinder the algorithm's performance by limiting the spread of information across the system. Thus, following [19], we introduced Parity RQAOA to circumvent this limitation, and we benchmarked the algorithm's single-layer implementation. We found that despite only requiring constant-depth quantum circuits, Parity RQAOA achieves the same approximation ratio as recursive vanilla QAOA, where the circuit depth scales linearly with the problem size.

The performance improvements achieved by RQAOA come at the cost of increased classical computational overhead, as the algorithm introduces multiple iterative optimization steps. Other QAOA variants [4, 12, 18] pursue a similar strategy by allocating additional classical resources to enhance performance at a fixed QAOA depth. As with RQAOA, they can in principle also be applied to parity-encoded QAOA [12]. Although a systematic numerical comparison with these variants is beyond the scope of this work, our results suggest that the characteristics of parity encodings observed for standard QAOA extend to more sophisticated and practically motivated QAOA variants.

Our findings indicate that Parity QAOA provides an alternative formulation to vanilla QAOA, with encouraging behavior observed in small-scale simulations. Furthermore, a lower-bound analysis motivates future studies at larger system sizes. As it has different demands on quantum hardware, it is essential to consider the specific characteristics of the target platform (2-qubit gate fidelity, coherence times, idle fidelity...) when selecting the appropriate implementation. In addition, the algorithm's performance strongly depends on the details of the implementation and the classical post-processing methods considered (e.g. decoding and recursive strategies). Therefore, future research should focus on further developing these techniques and extending them to a broader range of problems, including graphs with non-discrete weights. In these studies, classical simulations of Clifford circuits may offer resource-efficient strategies to compare different QAOA variants and architectures [12, 48, 55]. Moreover, the Clifford circuit initialization method proposed in [49] may provide low-energy starting points for QAOA, ultimately improving the algorithm's efficiency.

Finally, we note that the analysis presented does not account for quantum noise, which still plays a crucial role in current implementations of quantum algorithms. However, as reported in [35], we expect the redundant encoding of information in the Parity architecture and the spanning tree decoding to mitigate quantum noise, potentially making Parity QAOA less susceptible to gate errors and decoherence effects. A detailed investigation to verify these statements and test the algorithm's scalability on physical hardware remains an open area for further research.

Python code implementation of the Parity QAOA Clifford circuits considered in this work is available online [61].

Acknowledgments

We thank Philipp Aumann, Roeland ter Hoeven, Barry Mant, Anette Messinger, and Leo Stenzel for useful discussions. This study was supported by NextGenerationEU via FFG and Quantum Austria (FFG Project No. FO999896208). This research was funded in part by the Austrian Science Fund (FWF) under Grant-DOI 10.55776/F71 and Grant-DOI 10.55776/Y1067. This project was funded within the QuantERA II Programme that has received funding from the European Union's Horizon 2020 research and innovation programme under Grant Agreement No. 101017733. This publication has received funding under Horizon Europe programme HORIZON-CL4-2022-QUANTUM-02-SGA via the project 101113690 (PASQuanS2.1) This study was supported by the Federal Ministry for Economic Affairs and Climate Action through project QuaST. For the purpose of open access, the author has applied a CC BY public copyright license to any Author Accepted Manuscript version arising from this submission.

Data availability statement

The data cannot be made publicly available upon publication because the cost of preparing, depositing and hosting the data would be prohibitive within the terms of this research project. The data that support the findings of this study are available upon reasonable request from the authors.

Appendix A. Classical parameter vectors

A classical parameter vector $(\gamma, \Omega, \beta) \in A_{\text{class}}^{(p)}$ applied to QAOA unitary yields a classical state

$$|\tilde{q}\rangle = |\tilde{q}_1, \tilde{q}_2, \dots, \tilde{q}_K\rangle = \tilde{U}_x(\beta_p) \tilde{U}_{\text{CNOT}}(\Omega_p) \tilde{U}_z(\gamma_p) \times \dots \times \tilde{U}_x(\beta_1) \tilde{U}_{\text{CNOT}}(\Omega_1) \tilde{U}_z(\gamma_1) |+\rangle^{\otimes K}, \quad (20)$$

where $\tilde{q}_k \in \{0, 1\}$ with $k = 1, \dots, K$. For Parity QAOA and $p = 1$ we obtain four classical parameter vectors. Denoting those vectors with $\vartheta = (\gamma_1, \Omega_1, \beta_1)$, they are

$$\vartheta_1 = \left(\frac{\pi}{4}, 0, \frac{\pi}{4}\right), \quad (21)$$

$$\vartheta_2 = \left(\frac{\pi}{4}, \frac{\pi}{2}, \frac{\pi}{4}\right), \quad (22)$$

$$\vartheta_3 = \left(-\frac{\pi}{4}, 0, \frac{\pi}{4}\right), \quad (23)$$

$$\vartheta_4 = \left(-\frac{\pi}{4}, \frac{\pi}{2}, \frac{\pi}{4}\right). \quad (24)$$

Table 3 shows how to construct parameter vectors for $p > 1$. Here, we distinguish whether the number of layers p is even or odd. The index l and the values $(\gamma_l, \Omega_l, \beta_l)$ determine the whole set. For $p = 2$ and $p = 3$ one needs to skip some rows, namely the ones where a given sequence of index l is descending (first and third row for $p = 2$ and third row for $p = 3$).

Note that for $l > p/2$, there are two possible values for γ_l as well as for Ω_l if p is even, while for odd p there is an extra parameter γ_l at $l = (p+1)/2$. This leads to p parameters which can take two values at each p . The combination of all possible values comprises all 2^p classical parameter vectors at each p . Taking into account the parameter vectors for $q < p$ layers (by filling up the remaining values with zeros) and evaluating the partial sum of the resulting geometric series yields

$$|A_{\text{class}}^{(p)}| = \sum_{q=1}^p 2^q = \frac{1 - 2^{p+1}}{1 - 2} - 1 = 2^{p+1} \quad (25)$$

possible classical parameter vectors for p QAOA layers.

An example for constructing the classical parameter vectors for $p = 4$ (without accounting for the parameter vector for lower p) can be found in table 4.

Figure 8 shows that there is no significant difference in the approximation ratio r if all possible Clifford parameter vectors or if only Classical Clifford parameter vectors are used. This is due to the fact that the majority of Clifford parameter vectors return an equal superposition of all possible states. Neglecting those Clifford parameter vectors not only reduces the number of considered parameter vectors for the LB (both still scale exponentially in p) but also reduces the number of measurements needed. Classical Clifford vectors generate a circuit that returns a classical state, which requires a single measurement only, whereas a superposition state requires at least 1.000 measurements to estimate the energy.

Table 3. Classical parameter vectors for even (left) and odd (right) QAOA layers $p > 1$. The table shows the index values $l = 1, \dots, p$, for the parameters $(\gamma_l, \Omega_l, \beta_l)$ (given in units of π). For $p = 2$ and $p = 3$ descending indexes l need to be skipped. One can obtain all classical parameter vectors at layer p by combining the possible values for the $3p$ parameters.

even p		odd p	
l	$\gamma_l, \Omega_l, \beta_l$	l	$\gamma_l, \Omega_l, \beta_l$
$1, \dots, \frac{p-2}{2}$	$0, \frac{1}{4}, \frac{1}{4}$	$1, \dots, \frac{p-1}{2}$	$0, \frac{1}{4}, \frac{1}{4}$
$\frac{p}{2}$	$0, \frac{1}{4}, \frac{1}{2}$	$\frac{p+1}{2}$	$0, \frac{1}{2}, \frac{1}{4}, \frac{1}{4}$
$\frac{p+2}{2}, \dots, (p-1)$	$0, \frac{1}{2}, \pm \frac{1}{4}, \frac{1}{4}$	$\frac{p+3}{2}, \dots, (p-1)$	$0, \frac{1}{2}, \pm \frac{1}{4}, \frac{1}{4}$
p	$\pm \frac{1}{4}, \pm \frac{1}{4}, \frac{1}{4}$	p	$\pm \frac{1}{4}, \pm \frac{1}{4}, \frac{1}{4}$

Table 4. Classical parameter vectors for $p = 4$ layers (excluding the ones from $p < 4$). Values are given in units of π . The vectors are constructed by the method given in table 3. This table suggests two distinct values for the 4 parameters $\gamma_3, \Omega_3, \gamma_4$ and Ω_4 (bold symbols in the header). All possible combinations of the four values lead to a total of $2^4 = 16$ vectors. A change from bold numbers to normal ones and vice versa in the table indicates that the value is different than the one in the previous row. If there is no bold number in a column, the values are constant for all parameter vectors.

γ_1	Ω_1	β_1	γ_2	Ω_2	β_2	γ_3	Ω_3	β_3	γ_4	Ω_4	β_4
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	0	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	0	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	0	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	0	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	0	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	0	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	0	$-\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	0	$-\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
0	$\frac{1}{4}$	$\frac{1}{4}$	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{4}$	$-\frac{1}{4}$	$\frac{1}{4}$

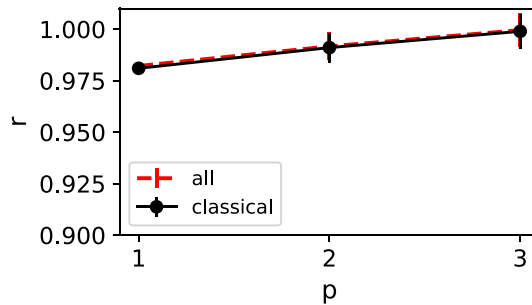


Figure 8. Approximation ratio achieved by using all possible Clifford angles (red dashed line) compared to using Classical Clifford parameters only (black solid line). Results shown are the average over 600 random generated instances of size $N = 7$. Errorbars indicate twice the standard error of the mean.

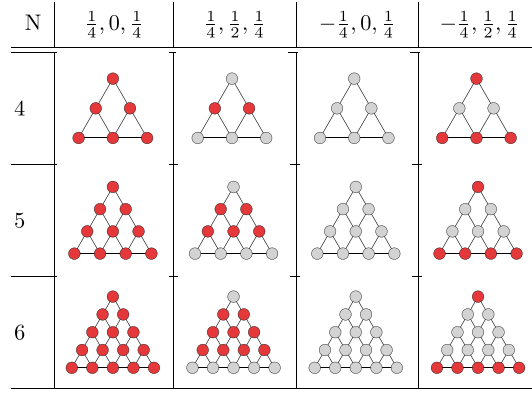


Figure 9. Classical parameter vectors in units of π for $p = 1$ with corresponding classical states for different logical qubit numbers N for a complete graph. Red (gray) circles denote qubits $\tilde{\sigma}_\nu^z$ that are anti-aligned (aligned) to their local field $\tilde{J}_\nu$. The first parameter set $(\frac{1}{4}, 0, \frac{1}{4})$ leads to the inverted local-field ground state, while the third set, $(-\frac{1}{4}, 0, \frac{1}{4})$ yields the local-field ground state. Also, the second and fourth sets differ by a global spin flip.

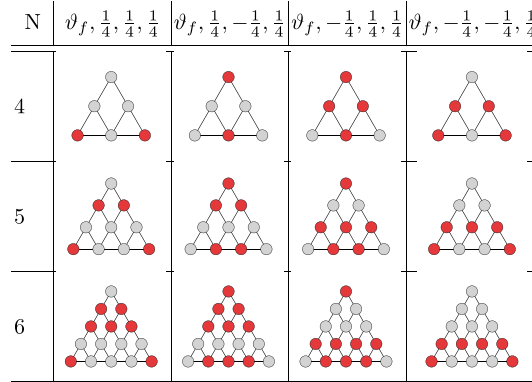


Figure 10. Classical states obtained from classical parameter vectors for $p = 2$ (excluding the ones already found for $p = 1$) for a complete graph. The values are given in units of π . All parameter vectors start with a fixed set $\vartheta_f = (\gamma_1, \Omega_1, \beta_1) = (0, \frac{1}{4}, \frac{1}{2})$. The values for $(\gamma_2, \Omega_2, \beta_2)$ are given in the header. Red (gray) circles denote qubits $\tilde{\sigma}_\nu^z$ that are anti-aligned (aligned) to the their field $\tilde{J}_\nu$. Negating γ_p leads to an inverted outcome. This is the same behavior as for $p = 1$.

Appendix B. Classical states

In this section, we focus on the classical states that are obtained from the circuits generated by the classical parameter vectors.

For $p = 1$, the four classical states $\tilde{q}_1$, $\tilde{q}_2$, $\tilde{q}_3$ and $\tilde{q}_4$ correspond to the parameter vectors from equations (21)–(24). The state $\tilde{q}_1$ is the ground state of the negative local-field Hamiltonian $-\tilde{H}_p$, equation (7). All qubits $\tilde{\sigma}_\nu^z$ are aligned to the fields $-\tilde{J}_\nu$. In contrast, the state $\tilde{q}_3$ has all qubits $\tilde{\sigma}_\nu^z$ aligned to the fields $+\tilde{J}_\nu$, indicating that $\tilde{q}_3$ is the ground state of the local-field Hamiltonian $\tilde{H}_p$. The states $\tilde{q}_1$ and $\tilde{q}_3$ differ by a global spin flip. Comparing the parameter vectors ϑ_1 and ϑ_3 shows that they differ by a sign in γ'_1 . In addition, both constraint values Ω'_1 are zero, which implies that both circuits perform only local rotations. Continuing to the sets ϑ_2 and ϑ_4 , which differ in the sign of γ_1 , their corresponding states $\tilde{q}_2$ and $\tilde{q}_4$ show the same behavior as the previous states and differ by a global spin flip. In more detail, for the state $\tilde{q}_2$, qubits that are in an uneven amount of constraints are aligned, while the rest is anti-aligned to the fields $\tilde{J}_\nu$. Figure 9 shows the configuration of the classical states for the signed Max-Cut problem on a complete graph and different system sizes N . Red (gray) circles denote qubits $\tilde{\sigma}_\nu^z$ that are anti-aligned (aligned) to the local field $\tilde{J}_\nu$.

Every classical state can be specified by the orientation of the qubits $\tilde{\sigma}_\nu^z$ to their field $\tilde{J}_\nu$. The pattern of aligned and anti-aligned qubits for a specific size, for example $N = 4$, can be extended to larger system sizes $N > 4$. Figure 10 displays the states occurring for $p = 2$, which shows that the state patterns become more complex for $p > 1$.

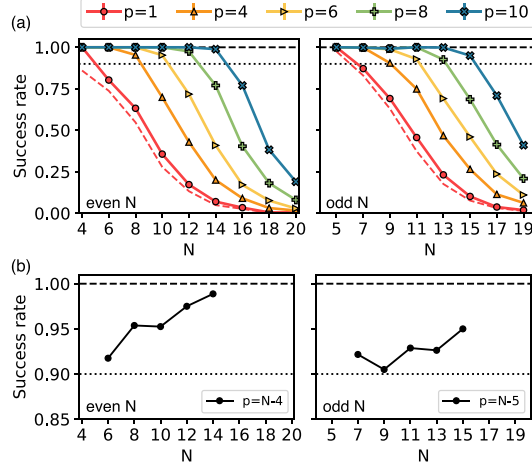


Figure 11. (a) Success rate of decoding the classical states of randomly generated problem instances for even (left) and odd (right) qubit numbers N with $M = N$ for different values of p for the signed Max-Cut problem on a complete graph. The red dashed line indicates the success rate of the ground state of $\tilde{H}_p$, equation (7) (or $\tilde{q}_3$). The horizontal black dotted line marks the success rate of 0.9. (b) Decoding success rate achieved when increasing p linearly with N (see legend in the subfigures). Results in each figure represent the averages over 50/200/400/600/800 simulated instances for $N = 4/5/6/7/ > 7$. The error bars, representing twice the standard error of the mean, are not visible due to their small size.

Appendix C. Decoding classical states

In this section, we address the question of how often decoding a classical state, obtained by the classical parameter vectors, yields the ground state for the signed Max-Cut problem on a complete graph. As outlined in the main text, we use the logical lines $T^{(l)}$ as spanning trees for decoding. We generate several problem instances with $J_{ij} \in \{-1, 1\}$ (exact numbers in the figure) and calculate the fraction of instances that were solved by decoding at $p \in \{1, 2, \dots, 10\}$. The results (for a selection of p) are shown in figure 11(a). To avoid displaying finite-size effects, which cause higher approximation ratios for odd compared to even qubit numbers, we split the data and show even qubit numbers in the left plot, and odd qubit numbers in the right one. The figure shows that decoding the classical states for $p = 1$ solves all problem instances for $N = 4$ and $N = 5$. That is, the success rate is equal to 1, marked with a horizontal dashed line in the plot. However, the success rate drops with increasing N . By increasing p , the success rate increases. In fact, if $p = N - 2$, all instances are solved for even qubit numbers. Interestingly, this does not hold for odd qubit numbers. There, more than 99% of the instances are solved when $p = N - 2$. In figures 11(a) and (b) the dotted horizontal line marks 90% success rate. The number of layers needed to obtain success rates beyond that threshold for even qubit numbers is $p = N - 4$ and for odd qubit numbers it is $p = N - 5$. The results for this very choice of p are plotted in figure 11(b), with even qubit numbers on the left and odd qubit numbers on the right. For even qubit numbers we observe the tendency of increasing success rate. The investigations were made for $p < 11$, hence, further studies should be done to check if this scaling persists for higher p and N .

For $N < 10$, more than half of the instances are solved by decoding the four classical states $\tilde{q}_1$, $\tilde{q}_2$, $\tilde{q}_3$ and $\tilde{q}_4$ that occur for $p = 1$. The red dashed line (without markers) in panel (a) marks the success rate for decoding the local-field ground state $\tilde{q}_3$. From the fact that the dashed line is not significantly lower than the red solid line (that combines all 4 states), we conclude that this state contributes most to the solved instances. In fact, for $N = 7$, more than 70% are solved by decoding the local-field ground state $\tilde{q}_3$. This state can be obtained by anti-aligning all spins σ_i^z with the local fields $\tilde{J}_i$. Indeed, the values for ϑ_3 , equation (23) imply that the constraint circuit is not applied as $\Omega'_1 = 0$. In [36], results of Parity QAOA simulations with the average spanning tree energy, defined in equation (12), as a objective function are shown. The parameters obtained there are the parameters ϑ_3 . Increasing the number of QAOA layers does not improve results as the local-field ground state s_3 coincides with the ground state of H_p for most instances with $N < 10$.

Appendix D. Implementation and resource comparison between Parity and Vanilla QAOA

In this section we elaborate on the differences between vanilla and Parity QAOA.

Implementation: Both, Vanilla and Parity QAOA require a compilation step. While for Vanilla QAOA, the problem graph is implemented directly on the hardware, necessitating a SWAP network if the graph's connectivity does not match the device topology, Parity QAOA requires Parity compilation [22]. For both compilations there will be a trade-off between compilation time and optimality of the compilation. For example, for Parity QAOA a 4-regular graph can be compiled within one step to the Parity triangle of the complete graph (figure 1), where all parity qubits representing non existing edges in the logical graph are ancillas. More compilation time will reduce ancillas and gates needed for implementation. The same holds for vanilla QAOA.

Qubit overhead: For a complete graph, the qubit overhead for Parity QAOA is quadratic in N , namely $K = \frac{N(N-1)}{2}$. For other graphs, it depends on the connectivity and whether ancillary qubits are needed. In this work, we execute vanilla QAOA on K/N copies of the initial state in parallel to account for the qubit overhead in Parity QAOA. After measurement of all qubits, the lowest energy obtained among all copies is used to determine the expectation value (see appendix E). This improves the approximation ratio over an implementation without copies. If one does not consider copies vanilla QAOA needs less qubits but more QAOA layers are needed to reach the same approximation ratio. This increases CNOT depth and gate count.

Post processing: For Parity QAOA we have M (spanning tree) read-outs per measurement of all qubits and keep the lowest energy to evaluate the objective function. For the complete graph we considered $M = N$ spanning trees and for the 4-regular graph the number of spanning trees was $\mathcal{O}(N)$. Similarly, for vanilla QAOA one obtains K/N states each shot (one for each copy), where, as in Parity QAOA, the state with the lowest energy is kept. The number of copies K/N depends on the qubit overhead of Parity QAOA which depends on the number of edges in the graph. For a complete graph the number of copies is the highest, namely $K/N = \frac{N-1}{2}$ and for the here studied 4-regular graph it is $K/N = 2$. For both implementations, increasing the number of readouts/copies will improve the results, but will also increase the classical post-processing resources.

A comparison of the resources needed to reach an approximation ratio $r > 0.95$ for both methods is shown in table 2. For the two small problems, the complete graph $N = 7$ and the 4-regular graph $N = 8$, the Parity QAOA circuit has a depth 5 times smaller than the vanilla QAOA circuit. For the complete graph with $N = 21$ we can only provide a LB for the approximation ratio of Parity QAOA. The QAOA circuit for this LB exhibits a depth 2 times smaller than the one of vanilla QAOA. In terms of CNOT gate count, the Parity QAOA circuit for the complete graph $N = 7$ comprises about 3.1 times fewer and the circuit for the 4-regular graph $N = 8$ has 2.5 times fewer gates. However, the Parity QAOA circuit for the complete graph with $N = 21$ includes about 3.6 times more CNOT gates (This is again an upper bound provided by Parity LB).

Appendix E. Simulation details

In this section, we provide details on the methods and parameters we used for obtaining the simulation results presented in the main text. The description of the classical optimizer is given on the example of vanilla QAOA but also holds for Parity QAOA. The classical optimization of the parameters (β, γ) starts with initializing the start parameters by choosing random values from the interval $[-\frac{\pi}{2}, \frac{\pi}{2})$ (and not from A as done in the Clifford pre-optimization described in section 2.4). The parameters are updated with the Monte-Carlo technique outlined below. A new parameter set $(\beta_{\text{new}}, \gamma_{\text{new}})$ is generated by updating one value, e.g. $\beta_1 \rightarrow \beta_1 + \kappa$, where κ is randomly drawn from a uniform distribution $\kappa \in [-0.25, 0.25]$. The new parameter vector is accepted if and only if

$$v < \exp\left(\frac{-\mathcal{C}(\beta_{\text{new}}, \gamma_{\text{new}}) + \mathcal{C}(\beta, \gamma)}{T}\right) \quad (26)$$

holds for a randomly generated value $v \in [0, 1]$. The effective temperature T is set to $T = 0.2$ in this work. After a certain amount of Monte-Carlo steps (update trials) n_{mc} , the optimization stops and we consider the lowest measured objective function $\mathcal{C}(\beta^*, \gamma^*)$ as the best outcome. For estimating the energy expectation for a given parameter set, the quantum circuit for these parameters is executed

Table 5. Number and percentage of Clifford-non-solvable instances with a given p by decoding classical states for a complete graph with $N = 7$ and $M = 7$ and a 4-regular graph with $N = 8$ and $M = 5$. For the latter also values for $M = 8$ are given in brackets. Numbers are for all possible interactions J_{ij} , that are 2^{15} instances for the complete graph and 2^{10} instances for the 4-regular graph.

p	no. of instances		% of total	
	Complete	4-regular	Complete	4-regular
5	36	6 (0)	0.11	0.59 (0.0)
4	94	20 (6)	0.28	1.95 (0.59)
3	1200	36 (10)	3.66	3.52 (0.98)

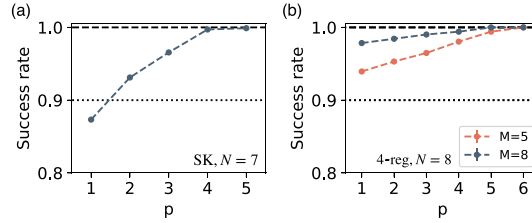


Figure 12. Success rate of decoding classical states of the signed Max-Cut problem on a (a) complete and a (b) 4-regular graph. The results shown are the average over all possible J_{ij} configurations for each problem graph. Exact numbers for $p > 2$ are given in table 5.

10.000 times. For every instance, n_{init} start parameters are initialized and updated with the previously described method. The best run is considered for later statistics. The selection for n_{init} and n_{mc} was made so that it ensures the simulation results are close to convergence—where further increasing the values would not lead to significant changes in the outcomes—while also balancing the need to minimize classical simulation time. To equalize the number of qubits needed in both QAOA approaches, we execute vanilla QAOA with K/N copies in parallel. For each shot the lowest energy of the copies is used to determine the expectation value.

The Clifford circuits are simulated using the library *stim*. In order to do so, the QAOA gates need to be decomposed into the following Clifford gates: $\tilde{U}_z(\pi/4)$ can be decomposed into $H_{xy}P_x$, where H_{xy} is a variant of the Hadamard gate that swaps the X and Y axes (instead of X and Z). $P_{\{x,y,z\}}$ are the $\{x,y,z\}$ -Pauli gates. A negative sign in the argument, $\tilde{U}_z(-\pi/4)$, swaps the order of the gates to P_xH_{xy} . $\tilde{U}_z(\pi/2)$ can be decomposed into P_yP_x . The constraint unitary $\tilde{U}_c$ is composed of CNOT chains (see [21, 33]) and a single z -Rotation that is applied in the same manner as $\tilde{U}_z$. $\tilde{U}_x(\pi/2)$ and $\tilde{U}_x(\pi/4)$ can be decomposed into P_zP_y and P_zH_{yz} , respectively.

Appendix F. QAOA for Clifford-non-solvable instances

In this section, we elaborate on the performance of Parity QAOA on Clifford-non-solvable problem instances—instances that are not solvable by classical state decoding.

F.1. Complete graph

For a complete graph, the maximal qubit number for Parity QAOA that is still feasible for classical simulation with *qiskit* is $N = 7$ ($K = 21$). With that amount of qubits, one can generate 2^{15} unique problem instances. An instance is said to be unique if it differs in its J_{ij} elements from others ($J_{ij} \neq J'_{ij}$) and, in addition, its local-field ground state $\tilde{q}_3$ violates different parity constraints than other instances. If $\tilde{q}_3$ and $\tilde{q}'_3$ violate the same constraints (but $J_{ij} \neq J'_{ij}$), the problems are not unique and will yield the same result when performing Parity QAOA under the same conditions. Therefore, the number of unique instances corresponds to the number of possible configurations of violating and fulfilling constraints. With $N = 7$, there are 15 constraints, leading to a total of 2^{15} possible constraint configurations.

Data from generating and decoding all classical states with the logical lines $T^{(l)}$ for $p = 5$ show that 36 instances are not solvable. For $p = 4$, the number of Clifford-non-solvable instances is 94 and for $p = 3$ it is 1200. An overview is given in table 5. Figure 12(a) shows the success rate of decoding classical states for $p \in \{1, 2, 3, 4, 5\}$.

Figure 13 shows results of QAOA simulations of Clifford-non-solvable instances for $p = 5$ (a) and $p = 4$ (b) (Results for $p = 3$ are in figure 4), whereas instances that were shown in one panel are not simulated again in another panel, e.g. panel (b) shows instances that are not solved at $p = 4$ and are not shown in (a). The parity representation of a complete graph is a triangle that is mirror-symmetric,

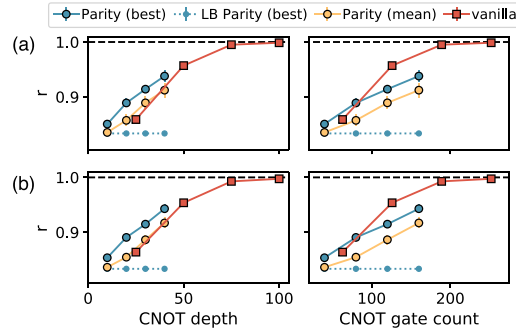


Figure 13. Approximation ratio r of Parity and vanilla QAOA for the complete graph of Clifford-non-solvable instances for (a) $p = 5$ and (b) $p = 4$ and $N = 7$ in dependence of CNOT depth and CNOT gate count. In each line, the left-most data point represents $p = 1$, the second left-most data point represents $p = 2$, and so forth. The data points show the average of 18/29 Clifford-non-solvable instances at $p = 5/p = 4$. That are all such instances (including the symmetry of the parity triangle) for each case whereas no instance is shown double, e.g. instances included in (a) are not considered in (b) again. The error bars indicate two times the standard error of the mean and are not visible due to the size of the markers. Classical optimization parameters are: $n_{\text{init}} = 80$, $n_{\text{mc}} = 400/1000/1500/2000$ for $p = 1/2/3/4$.

Table 6. Spanning trees used for decoding for the 4-regular graph given in figure 5. With 8 logical qubits, 7 interactions (parity qubits) form a spanning tree. The tuple (i, j) refers to the interaction between qubit i and qubit j , i.e. to the parity qubit ij .

t_1	(0, 2), (0, 1), (0, 6), (0, 7), (3, 7), (3, 5), (4, 5)
t_2	(0, 2), (0, 7), (5, 7), (4, 5), (4, 6), (3, 4), (1, 4)
t_3	(3, 6), (3, 7), (2, 7), (5, 7), (1, 5), (0, 1), (1, 4)
t_4	(2, 7), (2, 6), (1, 2), (0, 6), (3, 6), (1, 5), (4, 6)
t_5	(2, 6), (1, 2), (3, 4), (3, 5), (0, 6), (3, 6), (2, 7)
t_6	(0, 2), (0, 1), (2, 6), (1, 5), (5, 7), (3, 5), (4, 5)
t_7	(1, 2), (0, 7), (1, 4), (1, 5), (3, 4), (3, 7), (4, 6)
t_8	(1, 2), (0, 6), (0, 7), (0, 1), (3, 4), (3, 5), (3, 7)

which further reduces the number of unique problems. The exact numbers of unique Clifford-non-solvable instances for p layers can be found in figure 5. The results show that Parity QAOA when using $\tilde{C}^{\text{best}}$ as the objective function, has an advantage in terms of CNOT depth but not in CNOT gate count. This is in line with the results presented in section 3.2. The blue dotted line in each plot indicates the results obtained by decoding the classical states and represents a LB for the performance of Parity QAOA. Increasing the number of QAOA layers p (and thereby increasing circuit depth and gate count) does not improve the LB of the approximation ratio r . QAOA in the parity setting achieves an approximation ratio above the LB. With each additional layer, the approximation ratio r further improves. Using the mean spanning tree energy $\tilde{C}^{\text{mean}}$ for the objective function (equation (12)) yields a lower approximation ratio than using the best spanning tree energy $\tilde{C}^{\text{best}}$ (equation (13)). Note that, when the mean spanning tree energy is used as the objective function, we still use the best spanning tree energy to obtain the best result after the optimization, in accordance with the methods in [35]. Adding more layers increases the performance. This is in contrast to the results in [36] where Clifford-solvable instances (i.e. instances that are solvable by classical state decoding) make up a major part of the simulated instances.

Figure 14 shows the best parameters found by the classical optimization routine for Parity QAOA, with the $\tilde{C}^{\text{best}}$ objective function, for $p = 2$. Blue bars indicate the values of the first layer, $(\gamma_1^*, \Omega_1^*, \beta_1^*)$. For the constraint parameter Ω_1^* , the values concentrate at $\Omega_1^* = 0$. This is in line with [36]. However, Ω_2^* is nonzero. This suggests that one can skip the constraint gates for the first layer. Further work could investigate if starting with the local-field ground state s_3 instead of an equal superposition of all computational basis states leads to similar results.

F.2. 4-regular graph

Similar to the complete graph, we generate all unique problem instances for the 4-regular graph, shown in figure 5. This graph has 10 constraints in the parity mapping, leading to $2^{10} = 1024$ unique problem instances. The spanning trees for decoding are given in table 6. Note that the ancilla parity qubit 16 is never included. Further, we consider two different sets, one with 5 trees $T = \{t_1, t_2, t_3, \dots, t_5\}$ and one

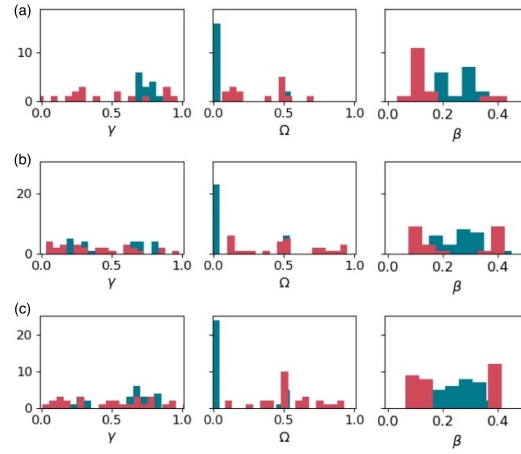


Figure 14. Histogram of the best parameters (in units of π) found in Parity QAOA using $\tilde{C}^{\text{best}}$, $p=2$ and $N=7$ corresponding to the problem cases in figures 13 and 4: (a) Clifford-non-solvable instances for $p=5$, (b) $p=4$ and (c) $p=3$. Blue indicates the parameters from the first ($\gamma_1^*, \Omega_1^*, \beta_1^*$), and red indicates the parameters for the second layer, ($\gamma_2^*, \Omega_2^*, \beta_2^*$).

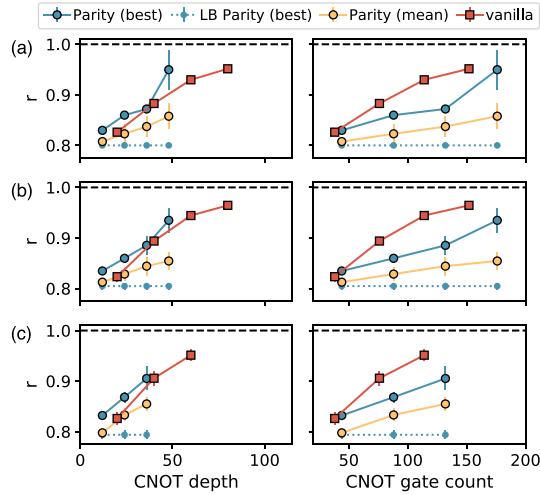
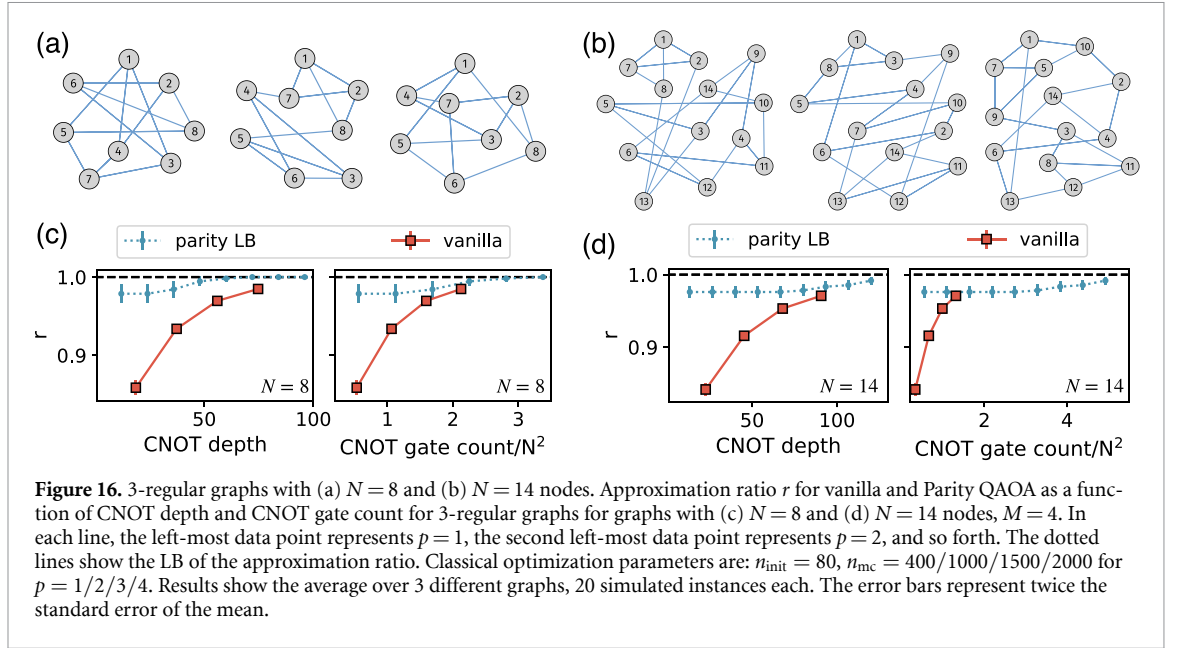


Figure 15. Approximation ratio r of Parity and vanilla QAOA for a 4-regular graph of all Clifford-non-solvable instances for (a) $p=5$, (b) $p=4$ and (c) $p=3$ and $M=5$ in dependence of CNOT depth and CNOT gate count. In each line, the left-most data point represents $p=1$, the second left-most data point represents $p=2$, and so forth. The data points show the average of 6/14/16 Clifford-non-solvable instances at $p=5/p=4/p=3$. That are all such instances for each case whereas no instance is shown double, e.g. instances included in (a) are not considered in (b) again. The error bars indicate two times the standard error of the mean and are not visible due to the size of the markers. Classical optimization parameters are: $n_{\text{init}} = 80$, $n_{\text{mc}} = 400/1000/1500/2000$ for $p = 1/2/3/4$.

with 8 trees $T = \{t_1, t_2, t_3, \dots, t_8\}$. The explicit numbers of Clifford-non-solvable instances for 5 spanning trees are given in table 5. Numbers for 8 spanning trees are given in brackets. Figure 12(b) shows the LB of the success rate of Parity QAOA for $p \in \{1, 2, \dots, 6\}$ for both $M=5$ and $M=8$. The success rate is higher for the case with $M=8$, showing that reading out more spanning trees increases the chance of decoding to the ground state. Both lines converge to 1, where the line for $M=8$ reaches this limit at $p=5$ while the line for $M=5$ reaches it at $p=6$. Due to the higher amount of Clifford-non-solvable instances, we perform QAOA simulations for those cases with $M=5$ spanning trees. The results are shown in figure 15. As before, we distinguish between Clifford-non-solvable instances at $p=5$ (a), at $p=4$ (b) and at $p=3$ (c). Instances that are shown in one panel are not considered again in another panels. In line with previous results, Parity QAOA with $\tilde{C}^{\text{best}}$ as objective function requires lower CNOT depth but a higher CNOT gate count. The LB of the approximation ratio for Parity QAOA does not increase with increasing layers but the performance of Parity QAOA does. Again, using the mean spanning tree energy $\tilde{C}^{\text{mean}}$ (equation (12)) as the objective function does not perform as well as the best spanning tree energy approach $\tilde{C}^{\text{best}}$ (equation (13)).



Appendix G. QAOA on more graph instances

In this section we show QAOA results for 3-regular graphs with $N=8$ and $N=14$ nodes, section G.1, 4-regular graphs with $N=14$ nodes, section G.2 and on a hypergraph with $N=8$ nodes, section G.3. For the 3- and 4-regular graph, we generate three different graphs for each size. The circuits for vanilla QAOA are compiled with t|ket> [27], where we consider the best out of 10 compilations. The parity circuits are compiled with ParityOS [22, 47]. Simulation details are the same as in the main text (appendix E).

G.1. 3-regular graphs

The three 3-regular graphs with $N=8$ and $N=14$ nodes used for simulations are shown in figure 16(a) and (b), respectively. The number of Parity qubits required to implement the graphs with $N=8$ nodes is $K=15$ (of which 3 are ancilla qubits). One of the graphs with $N=14$ nodes required $K=31$ Parity qubits (of which 10 are ancilla qubits) while the other two graphs with $N=14$ nodes required $K=41$ Parity qubits (of which 20 are ancilla qubits). We observe that, due to the 3-regular graphs' sparsity, multiple ancilla Parity qubits are required to build 3- and 4-body constraints in the parity implementation (see an example of an ancilla qubit in figure 5). Still, the qubit overhead is smaller than for a complete graph. Results for vanilla QAOA and Parity LB (using $M=4$ spanning trees for decoding) are shown in figure 16 for graphs of size (c) $N=8$ and (d) $N=14$. The small error bars (twice the standard error of the mean) indicate, that there is no stark difference between the three different graphs and their instances. As in the results for complete graphs presented in the main text, the Parity QAOA LB outperforms vanilla QAOA at fixed depth at all system sizes N .

G.2. 4-regular graphs

Here, we consider three 4-regular graphs with $N=14$ nodes, shown in figure 17(a). Implementing one graph requires $K=56$ Parity qubits, while the remaining two graphs require $K=54$ Parity qubits. Average approximation ratio over 20 instances for each graph are presented in figure 5(b). The results confirm the behavior observed in the main text for $N=8$ nodes, Parity QAOA achieves higher approximation ratio r with lower depth but the advantage in CNOT gate count disappears with increasing system size N . Comparing these results with those for the 3-regular graph of the same size shown in figure 16(d), we observe that the difference in CNOT gate count is less pronounced for 3-regular graphs than for 4-regular graphs. This is because 3-regular graphs require fewer Parity qubits and consequently fewer CNOT gates per layer of the Parity QAOA circuit.

G.3. Hypergraphs

Parity QAOA can be applied to QUBO problems (as described in the main text, section 2.3) and to higher-order binary optimization problems [22]. This section will focus on QAOA simulations for a problem with higher-order interaction terms, which is represented by the hypergraph shown in

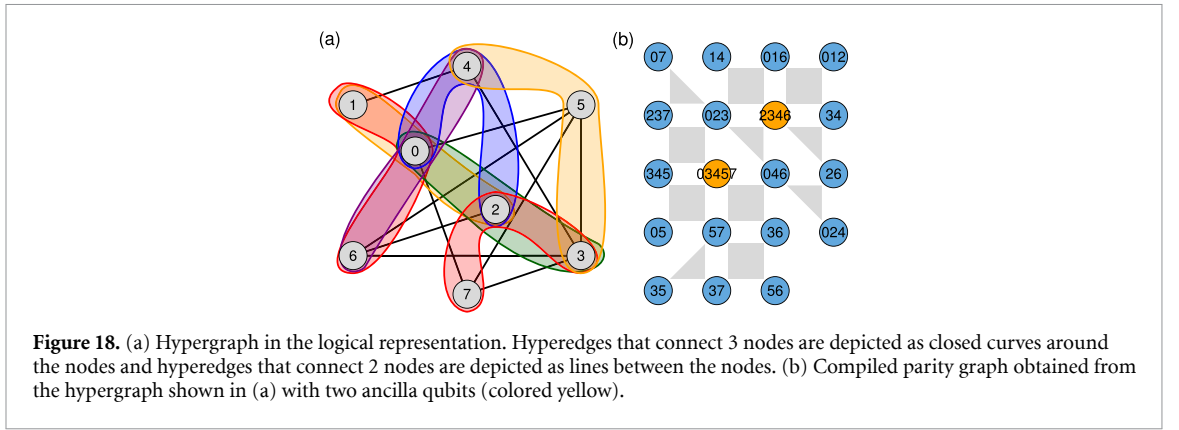
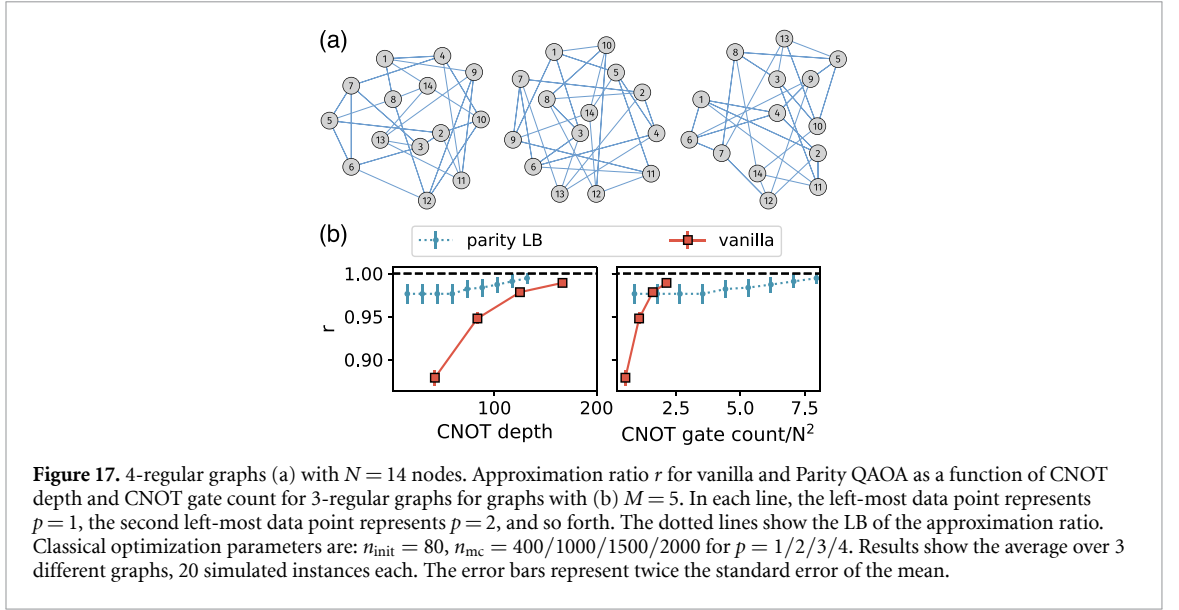


figure 18(a). The hypergraph consists of hyperedges that connect up to 3 nodes. Therefore, we modify equation (3) to take 3-body interaction into account and rewrite it as

$$H_P = \sum_{(i,j) \in E_2} J_{ij} \sigma_i^z \sigma_j^z + \sum_{(i,j,k) \in E_3} J_{ijk} \sigma_i^z \sigma_j^z \sigma_k^z, \quad (27)$$

where E_2 is the set of edges that connect 2 nodes, E_3 is the set of hyperedges that connect 3 nodes, and J_{ij} and J_{ijk} denote the weights for the corresponding hyperedges for the given $i, j, (k)$ nodes. In figure 18(a) hyperedges that connect 2 nodes are depicted as lines (10 in total), and hyperedges that connect 3 nodes are depicted as closed curves (7 in total). In figure 18(b), one can find the corresponding parity mapping that was obtained by parity compilation [22, 47].

In figure 19 we show the simulation results in terms of the approximation ratio for vanilla and Parity QAOA for this hypergraph. Here, Parity QAOA is simulated with the best spanning tree energy $\tilde{C}^{\text{best}}$, equation (13), as an objective function, using $M = 5$ readout bases obtained by the ParityOS package [47]. In this context, a readout basis is defined as a minimal set of physical qubits from which one can decode the configuration of the logical qubits and can be seen as a generalization of a spanning tree for hypergraphs. For the simulations, 100 problems were randomly generated by assigning random weights from $\{-1, +1\}$ to the coefficients J_{ij} and J_{ijk} for each hyperedge in equation (27). The QAOA simulations presented in this section were performed with qiskit [46]. For hypergraphs, we used the COBYLA optimizer from SciPy [62–65] instead of the Monte-Carlo optimizer employed in the main text, because we found it to be more efficient on this problem class.

The results show that for this hypergraph, Parity QAOA outperforms vanilla QAOA. Also, in contrast to the 4-regular graph case (see figure 6), Parity QAOA requires fewer resources in terms of both, CNOT count and CNOT depth. To be precise, Parity QAOA has a CNOT depth (gate count) of 13 (46) and vanilla QAOA has a CNOT depth (gate count) of 32 (61). The circuit for vanilla QAOA was optimized

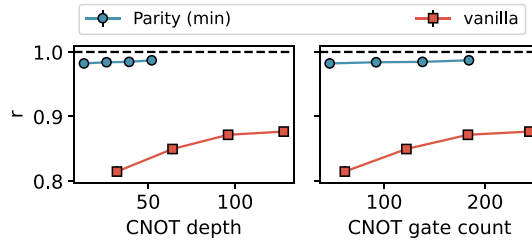


Figure 19. Approximation ratio r for the given hypergraph for vanilla and Parity QAOA as a function of the CNOT depth and CNOT gate count. In each line, the left-most data point represents $p = 1$, the second left-most data point represents $p = 2$, and so forth. Results show the average over 100 simulated instances. As a classical optimizer, we used COBYLA from SciPy [62–65], with 200 optimization steps, tolerance was set to 10^{-4} , and $n_{\text{init}} = 25/50/75/100$ for $p = 1/2/3/4$. The number of circuit runs for the objective function evaluation was set to 500.

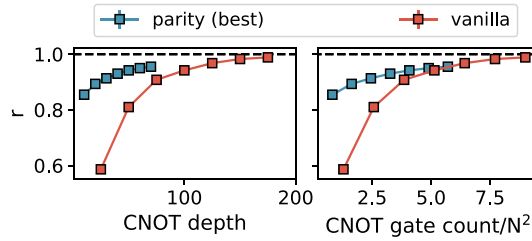


Figure 20. Approximation ratio r for LR-vanilla and LR-inspired-Parity QAOA as a function of CNOT depth and CNOT gate count. In each line, the left-most data point represents $p = 1$, the second left-most data point represents $p = 2$, and so forth. Results for $N = 7$ logical qubits ($K = 21$ physical qubits). For LR-vanilla we considered K/N copies. Results represent the average over 100 simulated instances (same as in figure 3), error bars represent twice the standard error of the mean.

with the qiskit [46] and tket compilers [27], where the qiskit compiler yielded better results, which we used for the simulations. The lower CNOT depth and lower gate count for Parity QAOA for this problem graph are in line with the results in [34], an advantage of Parity QAOA in those metrics for hypergraphs is anticipated.

Appendix H. Linear-ramp QAOA (LR-QAOA)

When the vanilla QAOA parameters follow a linear annealing schedule, good results can be achieved without the need of the classical optimization loop [53, 66]. There are no similar studies with Parity QAOA.

In the following, we will revise the linear-ramp protocol [53] and apply it to Parity QAOA on a complete graph with $N = 7$ nodes.

The Linear Ramp protocol determines the parameters for LR-vanilla QAOA in the following way

$$\beta_i = \left(1 - \frac{i}{p}\right) \Delta_\beta \text{ and } \gamma_i = \left(\frac{i+1}{p}\right) \Delta_\gamma, \quad (28)$$

with $i = 0, \dots, p-1$. By defining p , Δ_β and Δ_γ one gets a full parameter vector for a LR-vanilla QAOA run without needing the classical optimization loop [53].

For LR-Parity QAOA we can use the same parameters, but we need to define an additional parameter, namely Ω_i . We tested two cases, one where we follow the LR protocol and set $\Omega_i = \gamma_i$ and one LR-inspired approach with $\Omega_i = \gamma_i^2$, which was shown to lead to better results in an annealing process [67]. Following [53] we scan over a set of Δ_β and Δ_γ from one problem ($p = 7$) and use the best value over the remaining instances. For LR-vanilla QAOA we obtain $\Delta_\beta = 0.55$ and $\Delta_\gamma = -0.25$ and for LR-Parity QAOA and LR-inspired-Parity QAOA we get $\Delta_\beta = 0.3$ and $\Delta_\gamma = -0.25$. We obtain better approximation ratios when using the LR-inspired-Parity QAOA approach which is in accordance with the annealing results [67].

The simulation results are shown in figure 20. LR-vanilla QAOA approaches an approximation ratio of 1, however, it takes more layers than with the usual QAOA routine, see figure 3. As in the main text, we considered K/N copies. LR-inspired-Parity QAOA does not reach $r \approx 1$ (which can indicate that the

used schedule is not optimal for Parity QAOA) but we see an increase in r with increasing QAOA layers p . In general, we observe the same behavior as for standard QAOA: LR-inspired-Parity QAOA achieves higher r with less depth than LR-vanilla QAOA.

ORCID iDs

Anita Weidinger  0000-0001-8784-1205
 Glen Bigan Mbeng  0000-0002-6171-3229
 Michael Fellner  0000-0002-6980-5806
 Davit Khachatryan  0000-0003-0412-8768
 Wolfgang Lechner  0000-0003-3662-1020

References

- [1] Nielsen M A and Chuang I L 2010 *Quantum Computation and Quantum Information* (Cambridge University Press)
- [2] Montanaro A 2016 Quantum algorithms: an overview *npj Quantum Inf.* **2** 15023
- [3] Farhi E, Goldstone J, and Gutmann S 2014 A quantum approximate optimization algorithm
- [4] Blekos K, Brand D, Ceschini A, Chou C-H, Rui-Hao L, Pandya K and Summer A 2024 A review on quantum approximate optimization algorithm and its variants *Phys. Rep.* **1068** 1–66
- [5] Preskill J 2018 Quantum computing in the NISQ era and beyond *Quantum* **2** 79
- [6] Zhou L, Wang S-T, Choi S, Pichler H and Lukin M D 2020 Quantum approximate optimization algorithm: performance, mechanism and implementation on near-term devices *Phys. Rev. X* **10** 021067
- [7] Shaydulin R et al 2024 Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem *Sci. Adv.* **10** eadm6761
- [8] Cerezo M et al 2021 Variational quantum algorithms *Nat. Rev. Phys.* **3** 625–44
- [9] Harrigan M P et al 2021 Quantum approximate optimization of non-planar graph problems on a planar superconducting processor *Nat. Phys.* **17** 332–6
- [10] Pelofske E, Bärtschi A and Eidenbenz S 2023 Quantum annealing vs. QAOA: 127 qubit higher-order ising problems on NISQ computers *High Performance Computing* ed A Bhatele, J Hammond, M Baboulin and C Kruse (Springer) pp 240–58
- [11] Pellow-Jarman A, McFarthing S, Sinayskiy I, Park D K, Pillay A and Petruccione F 2024 The effect of classical optimizers and ansatz depth on QAOA performance in noisy devices *Sci. Rep.* **14** 16011
- [12] Herrman R, Lotshaw P C, Ostrowski J, Humble T S and Siopsis G 2022 Multi-angle quantum approximate optimization algorithm *Sci. Rep.* **12** 6781
- [13] Glos A, Krawiec A and Zimborás Z 2022 Space-efficient binary optimization for variational quantum computing *npj Quantum Inf.* **8** 39
- [14] Tabi Z, El-Safty K H, Kallus Z, Hága P, Kozsik T, Glos A and Zimborás Z 2020 Quantum optimization for the graph coloring problem with space-efficient embedding *2020 IEEE Int. Conf. on Quantum Computing and Engineering (QCE)* pp 56–62
- [15] Fuchs F G, Kolden H Ø, Aase N H and Sartor G 2021 Efficient encoding of the weighted max k -cut on a quantum computer using qaoa *SN Comput. Sci.* **2** 89
- [16] Bakó B, Glos A, Salehi Ö and Zimborás Z 2025 Prog-QAOA: framework for resource-efficient quantum optimization through classical programs *Quantum* **9** 1663
- [17] Hadfield S, Wang Z, O’Gorman B, Rieffel E G, Venturelli D and Biswas R 2019 From the quantum approximate optimization algorithm to a quantum alternating operator ansatz *Algorithms* **12** 34
- [18] Zhu L, Tang H L, Barron G S, Calderon-Vargas F A, Mayhall N J, Barnes E and Economou S E 2022 Adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer *Phys. Rev. Res.* **4** 033029
- [19] Bravyi S, Kliesch A, Koenig R and Tang E 2020 Obstacles to variational quantum optimization from symmetry protection *Phys. Rev. Lett.* **125** 260505
- [20] Lechner W, Hauke P and Zoller P 2015 A quantum annealing architecture with all-to-all connectivity from local interactions *Sci. Adv.* **1** e1500838
- [21] Lechner W 2020 Quantum approximate optimization with parallelizable gates *IEEE Trans. Quantum Eng.* **1** 1–6
- [22] Ender K, ter Hoeven R, Niehoff B E, Drieb-Schön M and Lechner W 2023 Parity quantum optimization: compiler *Quantum* **7** 950
- [23] Hirata Y, Nakanishi M, Yamashita S and Nakashima Y 2011 An efficient conversion of quantum circuits to a linear nearest neighbor architecture *Quantum Inf. Comput.* **11** 142–66
- [24] Alam M, Ash-Saki A and Ghosh S 2020 Circuit compilation methodologies for quantum approximate optimization algorithm *2020 53rd Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)* pp 215–28
- [25] Gushu Li, Ding Y and Xie Y 2019 Tackling the qubit mapping problem for nisy-era quantum devices *Proc. 24th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (New York, NY, USA) ASPLOS ’19 (Association for Computing Machinery)* pp 1001–14
- [26] Tan B and Cong J 2020 Optimal layout synthesis for quantum computing *2020 IEEE/ACM Int. Conf. on Computer Aided Design (ICCAD)* pp 1–9
- [27] Sivarajah S, Dilkes S, Cowtan A, Simmons W, Edgington A and Duncan R 2020 $t|ket\rangle$: a retargetable compiler for NISQ devices *Quantum Sci. Technol.* **6** 014003
- [28] Lao L and Browne D E 2022 2qan: a quantum compiler for 2-local qubit Hamiltonian simulation algorithms *Proc. of the 49th Annual Int. Symp. on Computer Architecture (New York, NY, USA), ISCA’22 (Association for Computing Machinery)* pp 351–65
- [29] Kotil A, Simkovic F and Leib M 2023 Improved qubit routing for qaoa circuits
- [30] Crooks G E 2018 Performance of the quantum approximate optimization algorithm on the maximum cut problem
- [31] Weidenfeller J, Valor L C, Gacon J, Tornow C, Bello L, Woerner S and Egger D J 2022 Scaling of the quantum approximate optimization algorithm on superconducting qubit based hardware *Quantum* **6** 870
- [32] Kivlichan I D, McClean J, Wiebe N, Gidney C, Aspuru-Guzik A, Chan G K-L and Babbush R 2018 Quantum simulation of electronic structure with linear depth and connectivity *Phys. Rev. Lett.* **120** 110501

- [33] Unger J, Messinger A, Niehoff B E, Fellner M, and Lechner W 2023 Low-depth circuit implementation of parity constraints for quantum optimization
- [34] Fellner M, Ender K, ter Hoeven R and Lechner W 2023 Parity quantum optimization: benchmarks *Quantum* **7** 952
- [35] Weidinger A, Mbeng G B and Lechner W 2023 Error mitigation for quantum approximate optimization *Phys. Rev. A* **108** 032408
- [36] Wybo E and Leib M 2024 Vanishing performance of the parity-encoded quantum approximate optimization algorithm applied to spin-glass models *Quantum* **8** 1554
- [37] Crowston R, Gutin G, Jones M and Muciaccia G 2013 Maximum balanced subgraph problem parameterized above lower bound *Theor. Comput. Sci.* **513** 53–64
- [38] Chiang C, Kahng A B, Sinha S, Xu X and Zelikovsky A Z 2007 Fast and efficient bright-field AAPSM conflict detection and correction *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **26** 115–26
- [39] DasGupta B, Enciso G A, Sontag E and Zhang Y 2007 Algorithmic and complexity results for decompositions of biological networks into monotone subsystems *Biosystems* **90** 161–78
- [40] Hüffner F, Betzler N and Niedermeier R 2007 Optimal edge deletions for signed graph balancing *Experimental Algorithms*, editor C Demetrescu (Springer) pp 297–310
- [41] Zaslavsky T 2018 A mathematical bibliography of signed and gain graphs and allied areas *Electron. J. Comb.* **DS8**
- [42] Garey M R and Johnson D S 1978 ‘strong’ np-completeness results: motivation, examples and implications *J. ACM* **25** 499–508
- [43] Garey M R, Johnson D S and Stockmeyer L 1976 Some simplified np-complete graph problems *Theor. Comput. Sci.* **1** 237–67
- [44] Polloreno A M and Smith G 2024 The qaoa with few measurements
- [45] Mbeng G B, Fazio R, and Santoro G 2019 Quantum annealing: a journey through digitalization, control, and hybrid quantum variational schemes
- [46] Qiskit contributors 2023 Qiskit: an open-source framework for quantum computing
- [47] Parity Quantum Computing GmbH 2024 ParityOS Available at PyPI
- [48] Muñoz Arias M H, Kourtis S and Blais A 2024 Low-depth Clifford circuits approximately solve MaxCut *Phys. Rev. Res.* **6** 023294
- [49] Cheng M H, Khosla K E, Self C N, Lin M, Li B X, Medina A C, and Kim M S 2022 Clifford circuit initialisation for variational quantum algorithms
- [50] Aaronson S and Gottesman D 2004 Improved simulation of stabilizer circuits *Phys. Rev. A* **70** 052328
- [51] Gottesman D 1997 Stabilizer codes and quantum error correction *PhD Thesis* California Institute of Technology
- [52] Gottesman D 1998 The Heisenberg representation of quantum computers
- [53] Montañez-Barrera J A and Michielsen K 2025 Toward a linear-ramp QAOA protocol: evidence of a scaling advantage in solving some combinatorial optimization problems *npj Quantum Inf.* **11** 131
- [54] Gidney C 2021 Stim: a fast stabilizer circuit simulator *Quantum* **5** 497
- [55] Wilkie A, Ostrowski J, and Herrman R 2024 An angle rounding parameter initialization technique for ma-qaoa
- [56] Vijendran V, Das A, Koh D E, Assad S M and Lam P K 2024 An expressive ansatz for low-depth quantum approximate optimisation *Quantum Sci. Technol.* **9** 025010
- [57] Farhi E, Gamarnik D, and Gutmann S 2020 The quantum approximate optimization algorithm needs to see the whole graph: a typical case
- [58] Bravyi S, Kliesch A, Koenig R and Tang E 2022 Hybrid quantum-classical algorithms for approximate graph coloring *Quantum* **6** 678
- [59] Finžgar J R, Kerschbaumer A, Schuetz M J A, Mendl C B and Katzgraber H G 2024 Quantum-informed recursive optimization algorithms *PRX Quantum* **5** 020327
- [60] Brady L T and Hadfield S 2023 Iterative quantum algorithms for maximum independent set: a tale of low-depth quantum algorithms
- [61] Parity qaoa with clifford circuits (available at: <https://gitlab.com/AnitaW/parity-qaoa-with-clifford-circuits.git>)
- [62] Virtanen P et al 2020 SciPy 1.0: fundamental algorithms for scientific computing in python *Nat. Methods* **17** 261–72
- [63] Powell M J D 1994 *A Direct Search Optimization Method That Models the Objective and Constraint Functions by Linear Interpolation* (Springer) pp 51–67
- [64] Powell M J D 1998 Direct search algorithms for optimization calculations *Acta Numer.* **7** 287–336
- [65] Powell M J D 2007 A view of algorithms for optimization without derivatives *Math. Today* **43** 170–4
- [66] Brandao F G S L, Broughton M, Farhi E, Gutmann S, and Neven H 2018 For fixed control parameters the quantum approximate optimization algorithm’s objective function value concentrates for typical instances
- [67] Susa Y and Nishimori H 2020 Performance enhancement of quantum annealing under the lechner–hauke–zoller scheme by non-linear driving of the constraint term *J. Phys. Soc. Japan* **89** 044006